

---

# Continual Learning of Domain-Invariant Representations

---

Pascal Janetzky<sup>1 2 3</sup> Tobias Schlagenhaut<sup>4</sup> Stefan Feuerriegel<sup>1 2</sup>

## Abstract

Continual learning (CL) aims to train models sequentially over multiple domains without forgetting previously learned knowledge. However, existing CL methods optimize for in-domain performance and are therefore prone to learning spurious, domain-specific cues (“shortcut learning”), which limits generalization to unseen domains after deployment. In this paper, we address this limitation through *continual learning of domain-invariant representation*. We introduce a broad class of CL methods that sequentially learn representations capturing invariant structures across domains. Our methods are motivated by the observation that such invariant structures often preserve the underlying causal mechanisms, which can reduce the risk of overfitting to domain-specific cues and thus offer better out-of-domain generalization. Our proposed CL methods combine replay-based training with a tailored sequential invariance alignment to learn—and preserve—invariant structures over time. We evaluate our methods under a deployment-oriented protocol that measures performance on unseen target domains. Across six benchmark and real-world datasets spanning vision, medicine, manufacturing, and ecology, our methods consistently outperform existing CL baselines in terms of generalization to unseen target domains. As an ablation, we further show that naïve extensions of sequential training with existing domain-invariant representation learning (DIRL) methods provide only limited benefits. To the best of our knowledge, this is the first work to develop domain-invariant representation methods for CL.

---

<sup>1</sup>LMU Munich, Munich, Germany <sup>2</sup>Munich Center for Machine Learning (MCML), Munich, Germany <sup>3</sup>Bosch Center for Artificial Intelligence (BCAI), Renningen, Germany <sup>4</sup>Albstadt-Sigmaringen University, Germany. Correspondence to: Pascal Janetzky <pascal.janetzky@bosch.com>.

## 1. Introduction

Continual learning (CL) trains machine learning models sequentially on multiple domains without forgetting previously learned knowledge (Parisi et al., 2019; Wang et al., 2024, see Section 2 for a detailed method overview). This setting arises in many real-world applications, including autonomous driving (Shaheen et al., 2022), robotics (Thrun & Mitchell, 1995), medicine (Vokinger et al., 2021; Bruno et al., 2025), and manufacturing (Hurtado et al., 2023), where models must be continually updated as new data becomes available. A key challenge is to prevent forgetting, often referred to as “catastrophic forgetting” (Ratcliff, 1990), as new information is learned, which is typically measured by the in-domain performance on held-out splits of the training domains (Chaudhry et al., 2019a; Caccia et al., 2021).

However, the primary objective in existing CL methods is to maintain performance on previously seen training domains (Kirkpatrick et al., 2017; Saha et al., 2021; Wang et al., 2024). This can encourage models to rely on spurious, domain-specific cues that are predictive within individual domains, but that do *not* reflect stable relationships shared across domains. Such behavior is closely related to “*shortcut learning*”, where models exploit superficial regularities that are sufficient for in-domain generalization but fail on new domains (Geirhos et al., 2020; Hermann et al., 2023). As a result, representations learned by standard CL methods may generalize poorly to new domains encountered after deployment.

This limitation is especially important in practice because many applications of CL are governed by biological (e.g. Lee & Lee, 2020; Kiyasseh et al., 2021; Bruno et al., 2025) or physical (e.g. Hurtado et al., 2023; Wang et al., 2023; Tang et al., 2024) processes, in which stable causal mechanisms determine the relationship between inputs and labels. For example, in medicine, physiological responses to radiotherapy follow the same underlying biological principles across patients, as radiation damages cancer cells through well-understood mechanisms. Similarly, in manufacturing, failure patterns are often driven by persistent physical processes, such as excessive heat, mechanical stress, or material fatigue, which remain consistent across machines, fabrication sizes, or production batches. Hence, these settings

motivate learning *domain-invariant representations*, which are representations that capture structures shared across domains while being insensitive to domain-specific variations, such as the background in images. For physical systems, these representations are known to improve performance (da Costa et al., 2020; Tanwani, 2021; Hua et al., 2025), as they are more likely to preserve stable causal mechanisms and, as a result, to support robust generalization to new domains. *However, to the best of our knowledge, methods for CL with domain-invariant representations are lacking.*

In this paper, we address the above limitation through **continual learning of domain-invariant representations**. We introduce a broad class of CL methods that sequentially learn representations while capturing invariant structures across domains, with the goal of improving generalization to unseen domains after deployment. We empirically show that learning such invariant representations is crucial for robust post-deployment performance.

① **Why continual learning of domain-invariant representation is non-trivial (Section 4).** Constructing CL methods that learn domain-invariant representations is challenging. Existing domain-invariant representation learning methods (e.g. Parascandolo et al., 2020; Krueger et al., 2021; Rame et al., 2022) are designed for *joint* access to multiple domains and then *simultaneously* optimize invariance constraints across domains; yet, this does *not* directly extend to sequential training. Here, domains arrive sequentially and past data are no longer fully accessible. As a result, naïve adaptations, such as replaying static invariance statistics or matching current representations to previously computed summaries, fail to faithfully reproduce multi-domain invariance objectives and offer little improvement over standard continual fine-tuning. We nevertheless include such naïve extensions in our experiments. While these adaptations have not been studied before in continual settings and are therefore novel in their own right, we later show that they provide only limited benefits over standard continual fine-tuning.

② **New CL methods for domain-invariant representations (Section 5).** To overcome the above challenges, we develop a broad class of CL methods that explicitly learn and preserve invariant structure over time. The proposed methods leverage replay-based training together with multi-domain invariance computation and sequential invariance alignment, which thus allows invariant structures to be maintained as new domains arrive. We consider multiple notions of domain invariance, namely, (i) risk-based, (ii) gradient-based, and (iii) feature-based—domain invariances. As a result, we propose five new CL algorithms: ★-CL-VREX, ★-CL-Fishr, ★-CL-CORAL, ★-CL-MMD, and ★-CL-ANDMask.

③ **Deployment-oriented evaluation protocol (Section 3).** We adopt a deployment-oriented evaluation protocol to

demonstrate generalization. Specifically, our evaluation mirrors the workflow in practical applications, given by *sequential training*  $\rightarrow$  *deployment*  $\rightarrow$  *testing on a new target domain* (Figure 1). This setting reflects real-world scenarios, such as in medicine or manufacturing, where models are trained on a sequence of source domains but where the characteristics of the eventual target domain are unknown at training time. Hence, we carefully measure the *out-of-domain performance* in addition to the in-domain performance that is typically reported in standard CL evaluations. Under this protocol, we conduct extensive experiments on six established benchmark and real-world datasets. Across all settings, our proposed methods consistently outperform existing CL baselines. This highlights that our proposed methods do not overfit to spurious, in-sample cues but actually learn domain-invariant mechanisms that generalize robustly to new domains.

**Contributions:** (1) We propose a set of tailored CL methods that learn domain-invariant representations over a sequence of source domains with the aim of generalizing to a new target domain. (2) We propose a deployment-oriented evaluation protocol to assess how methods generalize to new, previously unseen domains. (3) We perform extensive experiments across six datasets, showing that the proposed methods achieve SOTA performance. Therein, we further show that naïve extensions of sequential fine-tuning for domain-invariant learning have limited benefits, which motivates our proposed methods.

## 2. Related work<sup>1</sup>

**Continual learning (CL):** CL studies how models can be sequentially trained on a stream of data while preventing forgetting (McCloskey & Cohen, 1989). Most CL methods are designed around the stability-plasticity tradeoff (Lu et al., 2025) and are typically evaluated by how well they perform on held-out splits of previously seen training domains (Kirkpatrick et al., 2017; Saha et al., 2021; Wang et al., 2024). To achieve this, existing method broadly fall into four categories: (1) optimization-based methods, which modify gradient updates (Chaudhry et al., 2019a; Elsayed & Mahmood, 2024); (2) regularization-based methods, which reduce forgetting by regularizing updates to model weights or features (Kirkpatrick et al., 2017; Zenke et al., 2017; Magistri et al., 2024); (3) architecture-based approaches, which add a dedicated new network module for each new domain (and, hence, often scale quadratically or even exponentially with the number of domains) (Rusu et al., 2016; Yoon et al., 2017; 2019); and (4) replay-based approaches, which maintain a buffer to replay past samples (Chaudhry et al., 2019b; Churamani et al., 2023; Eskandar et al., 2025), features (Isken et al., 2020; Toldo & Ozay, 2022), or latent

<sup>1</sup>We provide an extended related work in Section A.

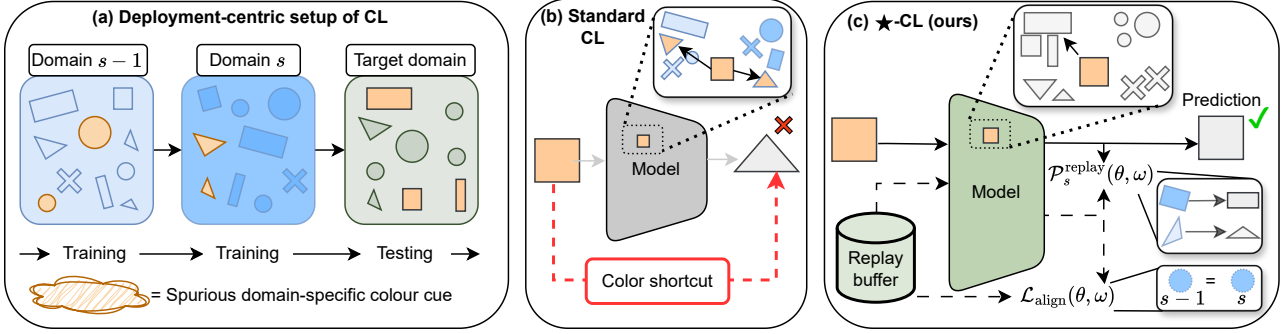


Figure 1. **Deployment-centric setup of CL.** (a): Training domains (shown in different colors) arrive sequentially; upon deployment, the model is evaluated on an arbitrary target domain. (b): Standard CL methods are prone to learning spurious, domain-specific cues (e.g., the color) and thus fail to classify data from unseen domains at deployment time. (c): Our methods learn domain-invariant representation (e.g., the shape) and thus generalize to both seen and unseen domains. Hence, our evaluation explicitly tests the out-of-domain generalization to ensure that robust patterns are learned, as in works on domain-invariant learning (Krueger et al., 2021; Rame et al., 2022).

information (De Lange & Tuytelaars, 2021; Sarfraz et al., 2023; 2025). Later, we benchmark our proposed methods against state-of-the-art CL methods, which serve as the main baselines in our CL setting.

However, the main goal of existing CL methods is to optimize performance on previously seen domains. This objective encourages learning patterns that are sufficient for high in-domain performance, including spurious, domain-specific cues that need not hold beyond the training distributions. As a result, models can achieve strong performance on all observed domains while still failing to generalize to new, unseen target domains after deployment. *In contrast, our work explicitly targets generalization beyond the training domains by learning domain-invariant structures over time.*

**Domain invariant representation learning (DIREL):** DIREL aims to learn features that transfer to new domains by enforcing invariances shared across multiple source domains (Li et al., 2018; Krishnamachari et al., 2024). As a result, invariant representations seek to encode the underlying causal mechanisms, which are stable across domains and which reduces overfitting to domain-specific cues and thus improves robustness to distribution shifts (Arjovsky et al., 2019; Krueger et al., 2021; Mahajan et al., 2021; Rame et al., 2022). Common examples are CORAL (Sun & Saenko, 2016) and MMD (Li et al., 2018) which align features across domains, VREX (Krueger et al., 2021), which aligns risks, and Fishr (Rame et al., 2022) and ANDMask (Parascandolo et al., 2020), which align gradients.

However, a key limitation of existing DIREL methods is that these assume *joint* access to multiple source domains during training to *simultaneously* optimize invariance constraints across domains (Sun & Saenko, 2016; Li et al., 2018; Rojas-Carulla et al., 2018; Arjovsky et al., 2019; Parascandolo et al., 2020; Shi et al., 2021; Rame et al., 2022; Krishna-

machari et al., 2024). This is *fundamentally different* from our CL scenario, where domains arrive *sequentially* and where past domains are no longer fully accessible. Hence, *standard DIREL methods are not directly applicable to CL without substantial modification.*

**Research gap:** To the best of our knowledge, there is currently **no** CL method that explicitly learns domain-invariant structures to achieve out-of-domain generalization. At the same time, DIREL methods are **not** directly applicable to CL settings, and, as we show later, naïve extensions provide only limited benefits, which motivate our tailored methods.

### 3. Problem setup

We follow standard CL literature (Serra et al., 2018; Kang et al., 2022; Sarfraz et al., 2025) and consider a supervised deployment-oriented setting where a model  $h$  is trained on a sequence  $\mathcal{S}$  of  $k$  labeled *source* domains  $D_s = \{\mathbf{x}_i^s, y_i^s\}_{i=1}^{n_s}$  and afterwards evaluated on an unseen *target* domain  $D_t = \{\mathbf{x}_i^t, y_i^t\}_{i=1}^{n_t}$ . We use the term ‘domain  $s$ ’ or ‘time  $s$ ’ interchangeably when referring to  $D_s$ . Each labeled domain  $D_\diamond, \diamond \in \{s, t\}$  has  $n_\diamond$  labeled samples  $\{(\mathbf{x}_i^\diamond, y_i^\diamond)\}_{i=1}^{n_\diamond}$ . All domains share the same label space, i.e.,  $y_i \in \{1, \dots, C\}$ , where  $C$  is the total number of classes. This setting is often referred to as domain-incremental learning (Van de Ven et al., 2022); but we evaluate on an unseen target domain.

We consider a deep neural network  $h$ , which consists of: (i) a *feature extractor*  $f_\theta : \mathcal{X} \rightarrow \mathcal{H}$ , which projects input data to latent space  $\mathcal{H}$ ; and (ii) a *classifier*  $g_\omega : \mathcal{H} \rightarrow \mathcal{Y}$ , which classifies the latent representations based on the set of predefined classes  $C$ . The full model is given by  $h = g_\omega \circ f_\theta = g_\omega(f_\theta(\cdot))$ .

Following standard practice in CL, training  $h$  proceeds sequentially over the domains. For each source domain  $D_s$ , the model parameters  $(\theta, \omega)$  are updated by minimizing the

empirical risk

$$\min_{\theta, \omega} \frac{1}{n_s} \sum_{i=1}^{n_s} \mathcal{L}(h(\mathbf{x}_i^s; \theta, \omega), y_i^s), \quad (1)$$

where  $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow [0, \infty)$  is a classification loss, such as the cross-entropy loss. This objective corresponds to standard empirical risk minimization (ERM) applied to the current domain  $s$ . In the continual learning setting, data from previous domains is typically no longer accessible once training moves on to a new domain. We therefore assume that only data from the current domain  $D_s$  is available at training time. Here, one typically allows for storing a small subset in a domain-partitioned *memory buffer*  $M_s$ , with  $|M_s| \ll |D_s|$ .

#### 4. Naïve domain-invariant CL algorithms

To learn domain-invariant representations along source domain sequence  $S$ , we first construct a set of naïve continual learning extensions of standard domain-invariant learning methods (e.g. Li et al., 2018; Krueger et al., 2021). For this, we decompose the training objective for domain  $s$  into an ERM term and an invariance penalty:

$$\min_{\theta, \omega} \mathcal{L}_{\text{ERM}}(D_s; \theta, \omega) + \lambda \mathcal{P}_s(\theta, \omega), \quad (2)$$

where  $\mathcal{L}_{\text{ERM}}(D_s; \theta, \omega) = \frac{1}{n_s} \sum_{i=1}^{n_s} \mathcal{L}(h(\mathbf{x}_i^s; \theta, \omega), y_i^s)$ , where  $\mathcal{P}_s$  is the invariance penalty for domain  $s$ , and where  $\lambda > 0$  is a scalar weighting factor.

**Invariance penalty:** To compute the invariance penalty, the naïve extensions maintain a set of domain-specific invariance priors  $\{\Phi_{s'}\}_{s' < s}$ , where each  $\Phi_{s'}$  is computed from  $D_{s'}$  at the end of training on domain  $s'$ .  $\Phi_{s'}$  thus stores domain-specific summary statistics and takes the form

$$\Phi_{s'} = \text{AGGREGATE}(\phi(\mathbf{x}, y; \theta, \omega) \text{ for } (\mathbf{x}, y) \sim D_{s'}) \quad (3)$$

where  $\phi(\cdot)$  is a function that extracts method-specific invariance information (e.g., feature moments (Sun & Saenko, 2016), risk (Krueger et al., 2021), gradient statistics (Rame et al., 2022)), and AGGREGATE denotes a running estimator using Welford’s method (Welford, 1962) to efficiently compute the average statistics for a domain.

**Training:** During training on minibatches from domain  $D_s$ , the naïve extensions penalize differences between current invariance statistics and stored priors  $\Phi_{s'}$  as

$$\mathcal{P}_s(\theta, \omega) = \frac{1}{s-1} \sum_{s'=1}^{s-1} d(\hat{\phi}_s(\theta, \omega), \Phi_{s'}), \quad (4)$$

where  $\hat{\phi}_s(\cdot)$  extracts method-specific invariance information from the current domain- $s$  batch, and  $d(\cdot, \cdot)$  is a distance

function such as  $\ell_2$ . The naïve extensions thus implement  $\mathcal{P}_s$  by comparing a domain- $s$  quantity (estimated from the current minibatch from domain  $s$ ) to *stored* quantities from previous domains. Intuitively, Eq. (4) treats each past domain as an “anchor” and encourages the current domain to match previously observed invariance statistics.

**Naïve extensions:** We construct the following five naïve extensions, which use different methodology to compute  $\hat{\phi}_s$  and  $\Phi_{s'}$ : (1) Naïve-CL-VREX, based on Krueger et al. (2021), which uses training risk as invariance information; (2) Naïve-CL-Fishr, based on Rame et al. (2022), which uses gradient statistics; (3) Naïve-CL-CORAL, based on Sun & Saenko (2016), which uses second-moment estimates from latent features; (4) Naïve-CL-MMD, based on Li et al. (2018), which uses Hilbert space kernel transformation on features; and (5) Naïve-CL-ANDMask, based on Parascandolo et al. (2020), which uses gradient sign agreement information. We provide further details in Section B.

As we empirically show later, these naïve algorithms achieve only minor improvements over standard continual fine-tuning. We attribute this to the fact that the stored static summaries can only *approximate* the real domain-invariance methods. The reason is that the original DIRM methods were designed for *joint* access to all source domains, which the summaries fail to faithfully reproduce. These limitations motivate our tailored methods

#### 5. Continual learning of domain-invariant representations

To overcome the limitations of the naïve methods, we propose a set of tailored methods that explicitly learn and preserve domain-invariant structure over time. Novel to our methods is that these (i) perform multi-domain invariance computation via replay and (ii) prevent the degradation of invariance information through domain-conditioned alignment. Overall, we provide five different methods (i.e., **★-CL-VREX**, **★-CL-Fishr**, **★-CL-CORAL**, **★-CL-MMD**, **★-CL-ANDMask**). An overview of our proposed methods is provided in Table 1.

##### 5.1. Components of our methods

Our proposed methods utilize the following components to sequentially learn domain-invariant representations: (1) memory buffers, (2) replay-augmented ERM, (3) invariance computation via replay, and (4) domain-conditioned invariance alignment. In the following, we detail each component, while the specific instantiation for each **★-CL** method is given in Table 1.

(1) **Memory buffer.** We maintain a memory buffer  $M$  with domain-conditioned partitions  $M_{s'}$  for each previ-

**Table 1. Our proposed set of CL methods** for sequentially learning domain-invariant representations. Each method defines a per-domain  $s'$  statistic  $\hat{\phi}_{s'}(\theta, \omega; B_{s'})$  computed on minibatch  $B_{s'}$  (current domain or replay), and a stored domain prior  $\Phi_{s'}$  computed at the end of domain  $s'$ . The invariance penalty  $\mathcal{P}_s^{\text{replay}}$  compares  $\{\hat{\phi}_{s'}\}_{e \leq s}$  *within the same update*; the alignment loss  $\mathcal{L}_{\text{align}}$  keeps replay-domain statistics representative of their historical domain.

| Method       | Per-domain stats & stored prior                                                                                                                                                                                                                                                                                                               | Invariance penalty $\mathcal{P}_s^{\text{replay}}$                                                                                                                                                                                                                                                           | Alignment loss $\mathcal{L}_{\text{align}}$                                                                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ★-CL-VREX    | $\hat{\phi}_{s'} \equiv \hat{r}_{s'} = \mathbb{E}_{(\mathbf{x}, y) \sim B_{s'}} [\mathcal{L}(h(\mathbf{x}; \theta, \omega), y)]$<br>$\Phi_{s'} \equiv \bar{r}_{s'}$ (domain-level average risk)                                                                                                                                               | $\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} (\hat{r}_{s'} - \bar{r})^2$ ,<br>with $\bar{r} = \frac{1}{s} \sum_{s' \leq s} \hat{r}_{s'}$                                                                                                                                                    | $\mathcal{L}_{\text{align}} = \sum_{e < s} (\hat{z}_{s'} - \bar{z}_{s'})^2$<br>with $\hat{z}_{s'} = h(\cdot; \theta_{s'}, \omega_{s'})$ and $\bar{z}_{s'} = h(\cdot, \theta_s, \omega_s)$ |
| ★-CL-Fishr   | $\hat{\phi}_{s'} \equiv \hat{\mathbf{v}}_{s'} = \text{Var}_{(\mathbf{x}, y) \sim B_{s'}} (\nabla_{\omega} \mathcal{L}(h(\mathbf{x}; \theta, \omega), y))$<br>$\Phi_{s'} \equiv \bar{\mathbf{v}}_{s'}$ (domain-level gradient-variance prototype via Welford's method)                                                                         | $\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} \ \hat{\mathbf{v}}_{s'} - \bar{\mathbf{v}}\ _2^2$ ,<br>with $\bar{\mathbf{v}} = \frac{1}{s} \sum_{s' \leq s} \hat{\mathbf{v}}_{s'}$                                                                                                            | $\mathcal{L}_{\text{align}} = \sum_{e < s} (1 - \cos(\log(\hat{\mathbf{v}}_{s'}), \log(\bar{\mathbf{v}}_{s'})))$                                                                          |
| ★-CL-CORAL   | $\hat{\phi}_{s'} \equiv (\hat{\mu}_{s'}, \hat{\Sigma}_{s'})$ with $\hat{\mu}_{s'} = \mathbb{E}_{\mathbf{x} \sim B_{s'}} [f_{\theta}(\mathbf{x})]$ and $\hat{\Sigma}_{s'} = \text{Cov}_{\mathbf{x} \sim B_{s'}} (f_{\theta}(\mathbf{x}))$<br>$\Phi_{s'} \equiv (\bar{\mu}_{s'}, \bar{\Sigma}_{s'})$ (batch-level feature mean and covariances) | $\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} (\ \hat{\mu}_{s'} - \bar{\mu}\ _2^2 + \ \hat{\Sigma}_{s'} - \bar{\Sigma}\ _F^2)$ ,<br>with $(\bar{\mu}, \bar{\Sigma}) = \frac{1}{s} \sum_{s' \leq s} (\hat{\mu}_{s'}, \hat{\Sigma}_{s'})$                                                      | $\mathcal{L}_{\text{align}} = \sum_{e < s} (\ \hat{\mu}_{s'} - \bar{\mu}_{s'}\ _2^2 + \ \hat{\Sigma}_{s'} - \bar{\Sigma}_{s'}\ _F^2)$                                                     |
| ★-CL-MMD     | $\hat{\phi}_{s'} \equiv \hat{\mu}_{s'}^z = \mathbb{E}_{\mathbf{x} \sim B_{s'}} [z(f_{\theta}(\mathbf{x}))]$ , with $z(\cdot)$ as random Fourier features for an RBF kernel<br>$\Phi_{s'} \equiv \bar{\mu}_{s'}^z$ (domain-level RFF mean embedding)                                                                                           | $\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} \ \hat{\mu}_{s'}^z - \bar{\mu}^z\ _2^2$ ,<br>with $\bar{\mu}^z = \frac{1}{s} \sum_{s' \leq s} \hat{\mu}_{s'}^z$                                                                                                                                | $\mathcal{L}_{\text{align}} = \sum_{e < s} \ \hat{\mu}_{s'}^z - \bar{\mu}_{s'}^z\ _2^2$                                                                                                   |
| ★-CL-ANDMask | Per-domain gradient $g_{s'} = \nabla_{\theta, \omega} \mathcal{L}_{\text{ERM}}(B_{s'}; \theta, \omega)$ ,<br>agreement mask $\mathbf{m} = \mathbb{I}(\frac{1}{s} \sum_{s' \leq s} \text{sign}(g_{s'})) \geq \tau$<br>with threshold $\tau \in [0, 1]$                                                                                         | <i>Implicit (via masked update):</i> $\nabla \leftarrow \mathbf{m} \odot \frac{1}{s} \sum_{s' \leq s} g_{s'}$ ,<br>with $g_{s'} = \nabla_{\theta, \omega} \mathcal{L}_{\text{ERM}}(B_{s'}; \theta, \omega)$<br>(i.e., $\mathcal{P}_s^{\text{replay}}$ is realized by enforcing cross-domain sign agreement). | $\mathcal{L}_{\text{align}} = \sum_{e < s} \mathbb{E}_{(\mathbf{x}, y, \mathbf{z}) \sim B_{s'}} [\ h(\mathbf{x}; \theta, \omega) - \mathbf{z}\ _2^2]$                                     |

ously seen source domain  $s' < s$ . Each stored element is a triple  $(\mathbf{x}, y, \mathbf{z})$  where  $\mathbf{z}$  are additional, method-specific information produced at insertion time. For example,  $\mathbf{z} = h(\mathbf{x}; \theta_{s'}, \omega_{s'})$  (logits) or  $\mathbf{z} = f_{\theta_{s'}}(\mathbf{x})$  (features), where  $(\theta_{s'}, \omega_{s'})$  denotes the model parameters at time  $s'$ . We later use  $\mathbf{z}$  for aligning invariances across time.

**② ERM with replay.** We expand the ERM term from Eq. (2) via

$$\mathcal{L}_{\text{ERM}}^{\text{replay}}(\theta, \omega) = \mathbb{E}_{(\mathbf{x}, y) \sim B} [\mathcal{L}(h(\mathbf{x}; \theta, \omega), y)], \quad (5)$$

where  $B$  is a joint data batch from the current domain  $s$ ,  $B_s \sim D_s$ , and replay domains  $s'$ ,  $B_{s'} \sim M_{s'}$  for  $s' = 1, \dots, s-1$ , constructed by  $B = \bigcup_{e \leq s} B_e$ .

**③ Multi-domain invariance computation via replay.** For each domain, we define the invariance penalty on the corresponding minibatch  $B_{s'}$ . Let  $\hat{\phi}_{s'}(\theta, \omega; B_{s'})$  denote the per-domain statistic (e.g., risk (Krueger et al., 2021), moments (Sun & Saenko, 2016), gradient statistics (Rame et al., 2022), etc.) computed on  $B_{s'}$ . We then instantiate the invariance penalty over  $\{B_{s'}\}_{s' \leq s}$ <sup>2</sup> as

$$\mathcal{P}_s^{\text{replay}}(\theta, \omega) = \text{INVPENALTY}(\{\hat{\phi}_{s'}(\theta, \omega; B_{s'})\}_{s' \leq s}) \quad (6)$$

which compares domain quantities *within the same update*, rather than via static loss priors.

**④ Domain-conditioned invariance alignment.** The previous Eq. (6) recovers within-step comparisons, but limited replay can make estimates noisy and can cause domain identity drift of replayed statistics as  $h$  evolves through

subsequent training. We therefore utilize knowledge distillation (Hinton et al., 2015) and reuse stored invariance priors  $\{\Phi_{s'}\}_{s' < s}$  from Eq. (3) to align replay-domain statistics to their original domain references:

$$\mathcal{L}_{\text{align}}(\theta, \omega) = \sum_{s' < s} d(\hat{\phi}_{s'}(\theta, \omega; B_{s'}), \Phi_{s'}), \quad (7)$$

where  $d(\cdot, \cdot)$  is a distance function as in Eq. (4). However, in contrast to Eq. (4), Eq. (7) does *not* anchor domain statistics to the past; instead, it keeps replayed samples representative of their historical domain by sampling batches  $B_{s'}$  from replay buffer  $M_{s'}$ .

**Overall objective.** Our overall objective combines replay-augmented ERM, sequential invariance penalties, and invariance alignment to give the generic training objective at domain  $s$ :

$$\min_{\theta, \omega} \mathcal{L}_{\text{ERM}}^{\text{replay}}(\theta, \omega) + \lambda \mathcal{P}_s^{\text{replay}}(\theta, \omega) + \beta \mathcal{L}_{\text{align}}(\theta, \omega), \quad (8)$$

where  $\lambda, \beta \geq 0$  are scalar weights.

## 5.2. Proposed methods

In total, we instantiate five different methods which differ in (i) the statistics  $\phi(\cdot)$ , (ii) the invariance operator  $\text{INVPENALTY}(\cdot)$ , and (iii) the alignment distance  $d(\cdot, \cdot)$ . Our methods are (1) **★-CL-VREX**, where  $\phi$  is the domain-wise empirical risk and  $\text{INVPENALTY}$  is the risk variance across domains; (2) **★-CL-Fishr**, where  $\phi$  is the per-domain gradient variance on  $\omega$  and  $\text{INVPENALTY}$  matches these variances across domains in the update; (3) **★-CL-CORAL**, where  $\phi$  uses feature moments  $(\mu_{s'}, \Sigma_{s'})$  that  $\text{INVPENALTY}$

<sup>2</sup>This includes the current domain by design, hence  $s' \leq s$ .

matches across domains; (4) **★-CL-MMD**, where  $\phi$  is a kernel mean embedding that  $\text{INVPENALTY}$  matches across domains; and (5) **★-CL-ANDMask**, where  $\phi$  is the set of per-domain gradients and  $\text{INVPENALTY}$  keeps only gradients whose signs agree across  $\{B_{s'}\}_{s' \leq s}$ .

For all methods, we chose the alignment term in Eq. (7) to match the corresponding invariance statistic, ensuring that replayed samples remain consistent with their original domain invariance signature. In Table 1, we provide a detailed overview of the methods and the method-specific equations. We provide minimal corresponding pseudocodes in Algorithm 1. We offer further details and extended pseudocodes in Section C and Algorithm 2, respectively.

**Algorithm 1** Pseudocode for our **★-CL methods**. See Alg. (2) for extended codes.

```

Input: Domain sequence  $S$ , memory buffer  $M$ 
1: for each domain  $D_s$  do
2:   for each batch in  $D_s \cup M$  do
3:     Compute  $\mathcal{L}_{\text{ERM}}^{\text{replay}}$ 
4:     Compute invariances via  $\mathcal{P}_s^{\text{replay}}$ 
5:     Align invariances to original reference via  $\mathcal{L}_{\text{align}}$ 
6:   end for
7: end for
    
```

## 6. Experimental setup

**Datasets.** We use six datasets that are commonly used in the literature (e.g., Koh et al., 2021; Shi & Wang, 2023; Liang & Li, 2023; Sarfraz et al., 2025): (1) RotatedMNIST (Ghifary et al., 2015), which is a variation of MNIST (LeCun, 1998) with multiple domains that correspond to different rotations of the original images. (2) CIFAR10C (Hendrycks & Dietterich, 2019) is a variation of CIFAR10 (Krizhevsky, 2009), but with multiple domains where each is subject to different corruptions of the input data. (3) TinyImageNetC (Hendrycks & Dietterich, 2019) is a similar modification of TinyImageNet (Le & Yang, 2015), where domains are created by applying different corruptions to the input pixels. (4) Camelyon17 (Bandi et al., 2019; Koh et al., 2021) is a real-world clinical image dataset of lymph node sections collected at different hospitals. (5) WM811K (Jang, 2015; Wu et al., 2015), a large-scale dataset from wafer manufacturing. (6) Covertype (Blackard, 1998), a dataset of cartographic and environmental features for predicting forest cover type. Details about the dataset, including source and target domain splits, are in Section E.

**Network architectures.** Following existing works in CL (Buzzega et al., 2020; Liang & Li, 2023), we use an ImageNet-pretrained ResNet18 (He et al., 2016) model for all experiments with large image datasets. For RotatedMNIST, we use a four-layer CNN following Rame et al. (2022). For Covertype, we use a four-layer MLP.

**Baselines.** We compare our proposed methods against 13 strong and widely used CL and three CDA/CTTA base-

lines: • *optimization-based methods*: **AGEM** (Chaudhry et al., 2019a) and **UPGD** (Elsayed & Mahmood, 2024); • *regularization-based methods*: **EWC** (Kirkpatrick et al., 2017), **SI** (Zenke et al., 2017), and **SNR** (Farias & Jozefiak, 2025); • *sample-replay-based methods*: **FDR** (Benjamin et al., 2019), **ER-ACE** (Caccia et al., 2021), **LODE** (Liang & Li, 2023), and **STAR** (Eskandar et al., 2025); • *latent-information replay-based methods*: **CoPE** (De Lange & Tuytelaars, 2021), **EFC** (Magistri et al., 2024), and **SARL** (Sarfraz et al., 2025); • *CDA/CTTA methods*: **TENT** (Wang et al., 2021), **SHOT++** (Liang et al., 2021), and **CoTTA** (Wang et al., 2022). We also benchmark against sequential fine-tuning, referred to as **Finetune**. As an upper-bound, we use **URM** (Krishnamachari et al., 2024) in a *offline setting*. URM has joint access to all source domains during training.

**Experimental settings.** We report the average classification accuracy (Acc) on the target domain  $\pm$  standard error across three runs. For WM811K, we report the macro F1 score  $\pm$  standard error. In comparisons, we report performance differences in %-points as *pp*. For replay-based methods, we follow prior work (Liang & Li, 2023) and set the memory buffer to 1000 (for RotatedMNIST and Covertype) or 5000 (remaining datasets) samples. Details about hyperparameter optimization are in Section F. All experiments are conducted on NVIDIA A100 and H100 GPU slices (MIG mode) with 32GB RAM and four CPU cores.

## 7. Results

① We first show the main experimental results for our proposed **★-CL** methods, followed by several ablation and robustness studies. ② We then demonstrate the subpar performance of the naïve extensions. ③ Afterwards, we show the improvements from our methods.

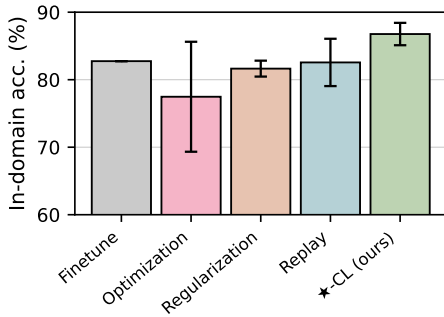
① **Main results.** We now perform the main experiment and benchmark our proposed methods against strong CL baselines. The results are in Table 2. We find: (1) Our methods achieve the first, second-, and third-best overall performance with 64.7%, 63.4%, and 63.1% accuracy / F1, respectively. (2) Our **★-CL** methods outperform optimization-, regularization-, and replay-based baselines by 6 pp, 10 pp, and 2 pp, respectively. (3) Our methods perform best in terms of rank: overall, the best method by rank are **★-CL-CORAL**, **★-CL-MMD**, **★-CL-VREX**.  $\Rightarrow$  **Takeaway:** *Our **★-CL** methods reach SOTA in terms of both average performance and average rank.*

② **Comparison against CDA/CTTA baselines.** We compare against CDA and CTTA methods in Table 5. For these baselines, we allow *unsupervised updates on the target domain*. Still, our proposed methods outperform the baselines by a large margin (up to 10pp).  $\Rightarrow$  **Takeaway:** *Our **★-CL***

**Table 2. Main results:** Comparison of **our proposed methods** against state-of-the-art CL baselines (**Finetune**, **optimization**-, **regularization**-, and **replay**-based methods) on six benchmark datasets. We report the mean $\pm$ standard error *target domain performance* across three independent runs. Results marked as **best**, **second**, **third**. “—” denotes *not applicable*.

| Method            | Accuracy / F1 $\uparrow$ ( $\uparrow$ ) |                                  |                                  |                                  |                                  |                                  |             | Rank ( $\downarrow$ ) |            |            |
|-------------------|-----------------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|-------------|-----------------------|------------|------------|
|                   | RotatedMNIST                            | CIFAR10C                         | TinyImageNetC                    | WM811K                           | Coverttype                       | Camelyon17                       | Avg         | Arith. mean           | Geom. mean | Median     |
| Finetune          | 39.5 $\pm$ 1.7                          | 66.3 $\pm$ 0.4                   | 18.5 $\pm$ 2.6                   | 82.8 $\pm$ 0.3                   | 8.1 $\pm$ 0.0                    | 86.9 $\pm$ 2.1                   | 50.4        | 14.3                  | 14.0       | 15.0       |
| AGEM              | 54.8 $\pm$ 4.7                          | 69.2 $\pm$ 0.2                   | 22.8 $\pm$ 1.1                   | 84.1 $\pm$ 0.2                   | 29.0 $\pm$ 8.5                   | 91.0 $\pm$ 0.1                   | 58.5        | 7.5                   | 6.2        | 8.5        |
| UPGD              | 29.1 $\pm$ 7.7                          | 53.9 $\pm$ 4.6                   | 19.6 $\pm$ 2.2                   | 74.6 $\pm$ 1.1                   | 23.4 $\pm$ 9.6                   | 88.7 $\pm$ 1.0                   | 48.2        | 16.0                  | 15.8       | 16.5       |
| EWC               | 42.5 $\pm$ 1.3                          | 66.7 $\pm$ 1.3                   | 20.1 $\pm$ 0.6                   | 84.2 $\pm$ 0.4                   | 8.0 $\pm$ 0.2                    | 90.7 $\pm$ 0.6                   | 52.0        | 10.8                  | 9.8        | 10.5       |
| SI                | 38.8 $\pm$ 0.7                          | 64.3 $\pm$ 2.4                   | 20.8 $\pm$ 1.2                   | 83.3 $\pm$ 0.6                   | 24.5 $\pm$ 6.8                   | 86.0 $\pm$ 3.6                   | 52.9        | 14.2                  | 14.1       | 13.5       |
| SNR               | 40.4 $\pm$ 0.2                          | 62.9 $\pm$ 1.6                   | 22.0 $\pm$ 1.7                   | 76.2 $\pm$ 0.6                   | 25.4 $\pm$ 10.4                  | 89.7 $\pm$ 0.9                   | 52.8        | 13.3                  | 13.0       | 14.0       |
| FDR               | 68.6 $\pm$ 1.3                          | 65.0 $\pm$ 0.9                   | 23.0 $\pm$ 1.0                   | 84.0 $\pm$ 0.4                   | 38.3 $\pm$ 0.7                   | 88.5 $\pm$ 1.0                   | 61.2        | 8.5                   | 8.1        | 8.5        |
| ER-ACE            | 67.5 $\pm$ 2.0                          | 66.9 $\pm$ 1.1                   | <b>29.0 <math>\pm</math> 1.2</b> | 85.4 $\pm$ 0.4                   | 37.7 $\pm$ 0.6                   | 90.3 $\pm$ 1.5                   | 62.8        | 5.2                   | 4.1        | 6.5        |
| LODE              | 60.0 $\pm$ 2.1                          | 64.7 $\pm$ 0.6                   | 21.6 $\pm$ 2.3                   | 80.1 $\pm$ 1.7                   | 37.5 $\pm$ 1.1                   | 65.2 $\pm$ 5.1                   | 54.9        | 12.5                  | 12.1       | 11.5       |
| STAR              | 67.4 $\pm$ 1.4                          | <b>69.5 <math>\pm</math> 0.3</b> | 25.6 $\pm$ 1.5                   | 85.1 $\pm$ 0.7                   | 38.3 $\pm$ 1.6                   | 86.9 $\pm$ 2.7                   | 62.1        | 6.5                   | 4.8        | 5.5        |
| COPE              | 68.7 $\pm$ 3.2                          | 64.2 $\pm$ 0.7                   | 20.1 $\pm$ 4.9                   | 84.0 $\pm$ 0.7                   | 9.4 $\pm$ 1.4                    | 90.6 $\pm$ 0.7                   | 56.2        | 11.0                  | 10.0       | 12.0       |
| EFC               | 65.8 $\pm$ 3.1                          | 66.7 $\pm$ 0.8                   | 21.3 $\pm$ 2.7                   | 84.7 $\pm$ 1.3                   | 25.8 $\pm$ 9.1                   | 90.9 $\pm$ 0.8                   | 59.2        | 8.3                   | 7.7        | 9.0        |
| SARL              | 63.9 $\pm$ 1.8                          | 62.9 $\pm$ 0.8                   | 0.7 $\pm$ 0.1                    | 75.5 $\pm$ 1.4                   | 41.2 $\pm$ 5.6                   | 79.6 $\pm$ 4.6                   | 54.0        | 13.7                  | 11.8       | 16.5       |
| ★-CL-VREX         | 70.2 $\pm$ 5.8                          | 67.2 $\pm$ 1.4                   | 26.3 $\pm$ 1.2                   | 84.1 $\pm$ 0.3                   | 40.8 $\pm$ 1.5                   | 91.5 $\pm$ 0.6                   | 63.4        | 4.3                   | 4.0        | 4.0        |
| ★-CL-Fishr        | 68.6 $\pm$ 0.4                          | 64.2 $\pm$ 1.2                   | 29.0 $\pm$ 2.2                   | 83.6 $\pm$ 0.5                   | 27.0 $\pm$ 7.7                   | 89.7 $\pm$ 2.0                   | 60.3        | 9.2                   | 7.8        | 10.0       |
| ★-CL-CORAL        | <b>72.8 <math>\pm</math> 2.6</b>        | 68.5 $\pm$ 1.7                   | 25.0 $\pm$ 2.8                   | 84.8 $\pm$ 0.6                   | <b>45.2 <math>\pm</math> 3.9</b> | <b>91.7 <math>\pm</math> 0.4</b> | <b>64.7</b> | <b>2.8</b>            | <b>2.1</b> | <b>2.5</b> |
| ★-CL-MMD          | 70.7 $\pm$ 0.5                          | 69.0 $\pm$ 0.5                   | 25.8 $\pm$ 0.8                   | <b>85.5 <math>\pm</math> 0.4</b> | 37.6 $\pm$ 0.9                   | 90.1 $\pm$ 0.5                   | 63.1        | 4.5                   | 3.6        | 3.5        |
| ★-CL-ANDMask      | 71.4 $\pm$ 2.6                          | 64.7 $\pm$ 0.7                   | 11.8 $\pm$ 3.6                   | 84.6 $\pm$ 0.6                   | 43.7 $\pm$ 2.9                   | 89.1 $\pm$ 1.5                   | 60.9        | 8.3                   | 6.1        | 8.5        |
| URM (Upper Bound) | 81.3 $\pm$ 1.4                          | 70.6 $\pm$ 0.7                   | 31.8 $\pm$ 0.3                   | 91.3 $\pm$ 0.4                   | —                                | 93.1 $\pm$ 0.8                   | 73.3        | —                     | —          | —          |

$\uparrow$  macro F1 for WM811K

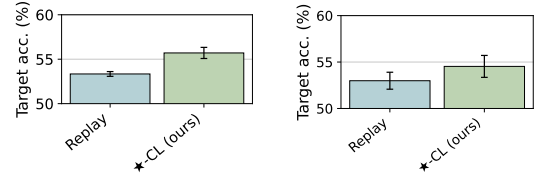


**Figure 2. In-domain performance.** Average across six datasets, per method group. The large std. err. (whisker) for **optimization** is caused by UPGD, which reaches a low in-domain performance. Our proposed methods outperform **Finetune** and strong **regularization** and **replay CL** baselines.

methods outperform CDA and CTTA baselines.

• **In-domain performance.** We report the in-domain performance in Figure 2, with details in Section G.8. We find: (1) Our proposed methods outperform strong CL baselines. (2) Nearly all methods attain sufficient in-domain performance, i.e., they have similar “starting conditions”. The better *out-of-domain* generalization from our methods must thus come from their ability to better learn the underlying causal mechanisms. **Takeaway:** *The use of DIRM benefits in-domain performance.*

• **Forgetting.** We report forgetting, as measured through the common backwards transfer (BWT) metric (Lopez-Paz & Ranzato, 2017), in Table 8. Negative numbers imply forgetting; positive numbers indicate retrospective improvement (which is desired). The results show that our proposed



**Figure 3. Reduced buffer sizes.** Average results for 50 % (left) and 25 % (right) memory capacity, providing less domain information. **Our methods** can nonetheless learn domain-invariant representations and outperform strong **replay-baselines**.

methods retain or retrospectively improve the performance on previous domains. **Takeaway:** *Our methods retrospectively improve the performance.*

• **Runtime analysis.** We provide runtimes in Table 9. The runtimes for our methods are in line with existing CL methods. **Takeaway:** *Runtimes are similar to the baselines, yet our methods generalize better to unseen domains.*

• **Ablation: invariance alignment.** We here study the effect of  $\mathcal{L}_{\text{align}}$  via (1) disabling alignment by setting  $\beta = 0$ , and (2) dynamically recomputing  $\Phi_{s'}$  at the end of each domain. The results are in Table 10 and Table 11. We find that aligning the invariance statistics over time is important for the final performance. This is an interesting finding, as aligning statistics to old domains is often used to *prevent forgetting*. However, our results indicate that aligning invariance statistics is equally important for *generalization* to unseen domains.  $\Rightarrow$  **Takeaway:** *Aligning invariance statistics over time improves generalization to unseen domains.*

• **Sensitivity: reduced buffer sizes.** Prior work in CL has shown that, for replay-based methods, the buffer size

strongly influences the final performance (e.g., [Aljundi et al., 2019a;b](#)), where larger buffers lead to better performance. We thus now make the problem *more challenging* and reduce the buffer capacity to 50 % and 25 %, respectively. We benchmark our proposed methods against best-performing replay-methods (FDR, ER-ACE, and STAR). To ensure fair comparison, we perform new hyperparameter searches. The results are in Figure 3 and Tables 12 and 13. We observe: (1) Our methods again reach the best performance, both in terms of average performance and position in the ranking. (2) Our proposed methods outperform strong replay baselines by  $\sim 4$  pp. (3) Our findings are consistent across the different buffer sizes.  $\Rightarrow$  **Takeaway:** *Our methods are robust to the memory buffer size and consistently outperform strong CL baselines.*

• **Robustness: different target domain.** We now exchange the dataset-specific target domain. We benchmark our proposed methods against baselines AGEM, UPGD, EWC, SI, FDR, and STAR. Note that, for fairness, exchanging the target domain necessitates that we again perform a new hyperparameter search. The results are in Table 15, details in Section G.9. Our methods again outperform optimization-based (UPGD, AGEM) and regularization-based (EWC and SI) methods and reach the best overall average performance.  $\Rightarrow$  **Takeaway:** *Our methods outperform existing CL baselines on different target domains.*

• **Robustness: different compute budget.** We now make learning robust representations even more challenging by reducing the number of training steps per domain by 50%, which also limits the number of samples per domain. We benchmark against AGEM, UPGD, EWC, SI, FDR, and STAR. As before, we perform a new hyperparameter study. The results are in Table 16. Our proposed methods outperform Finetune, optimization- and regularization-based methods by a clear margin and reach the best overall performance. The results show that, under reduced compute budgets, our methods can sequentially learn representations that better generalize to unseen target domains.  $\Rightarrow$  **Takeaway:** *Our results are robust to different compute budgets.*

• **Visualizations** We visualize the  $\lambda, \beta$ -hyperparameter sensitivity in Figures 8 and 9. We observe that our proposed methods are relatively robust to changes. Further, in Figures 10 and 11, we plot the feature space learned by our methods. We observe a **high representational overlap between the source and (unseen) target domains**, indicating that our methods successfully learn domain-invariant representations.

② **Naïve extensions fail to meaningfully outperform Finetune baseline.** We benchmark the naïve extensions (labeled by Naïve-CL) against Finetune and a subset of the baselines (i.e., best-performing ones per each category; full results are in Table 6), and evaluate on an unseen target

Table 3. Results for **naïve extensions**, which highlights the limited improvement over the **default sequential training** baseline. Further, naïve methods underperform baselines **AGEM**, **ER-ACE**, and **SI**. Shown: mean  $\pm$  standard error target domain accuracy across three independent runs. Marked: **best**, **second**, **third**.

| Method           | Accuracy ( $\uparrow$ )          |                                  |                                  |             | $\Delta$ ( $\uparrow$ )<br>to Finetune |
|------------------|----------------------------------|----------------------------------|----------------------------------|-------------|----------------------------------------|
|                  | RotatedMNIST                     | CIFAR10C                         | Coverttype                       | Avg         |                                        |
| Finetune         | 39.5 $\pm$ 1.7                   | 66.3 $\pm$ 0.4                   | 8.1 $\pm$ 0.0                    | 38.0        | +0.0                                   |
| AGEM             | 54.8 $\pm$ 4.7                   | <b>69.2 <math>\pm</math> 0.2</b> | 29.0 $\pm$ 8.5                   | 51.0        | +13.0                                  |
| SI               | 38.8 $\pm$ 0.7                   | 64.3 $\pm$ 2.4                   | 24.5 $\pm$ 6.8                   | 42.5        | +4.5                                   |
| ER-ACE           | <b>67.5 <math>\pm</math> 2.0</b> | 66.9 $\pm$ 1.1                   | <b>37.7 <math>\pm</math> 0.6</b> | <b>57.4</b> | <b>+19.4</b>                           |
| Naïve-CL-VREX    | 45.8 $\pm$ 6.8                   | 67.4 $\pm$ 0.3                   | 8.1 $\pm$ 0.1                    | 40.4        | +2.4                                   |
| Naïve-CL-Fishr   | 41.4 $\pm$ 1.5                   | 67.7 $\pm$ 0.3                   | 8.2 $\pm$ 0.1                    | 39.1        | +1.1                                   |
| Naïve-CL-CORAL   | 50.2 $\pm$ 5.3                   | 66.3 $\pm$ 0.8                   | 8.0 $\pm$ 0.1                    | 41.5        | +3.5                                   |
| Naïve-CL-MMD     | 39.9 $\pm$ 1.0                   | 66.3 $\pm$ 0.8                   | 7.8 $\pm$ 0.3                    | 38.0        | +0.0                                   |
| Naïve-CL-ANDMask | 21.5 $\pm$ 4.5                   | 26.9 $\pm$ 1.4                   | 6.6 $\pm$ 1.2                    | 18.3        | -19.7                                  |

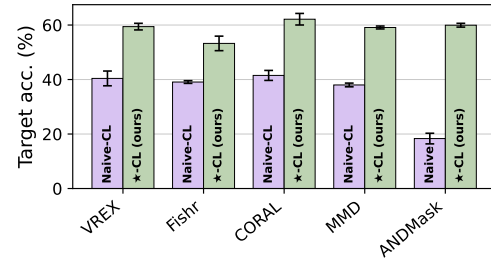


Figure 4. **Improvement from ★-CL over Naïve-CL.** Our proposed methods outperform the naïve extensions across all underlying methodologies for computing domain-invariant representations. Same datasets as in Table 3.

domain. The results are in Table 3. We observe: (1) The naïve extensions improve only slightly over the Finetune baseline by  $\sim 3$  pp, and (2) are considerably outperformed by the baselines SI and ER-ACE. These findings highlight the need for more tailored methods that learn domain-invariant representations in CL.  $\Rightarrow$  **Takeaway:** *Naïve extensions for domain-invariant learning in CL offer only minimal benefits over standard sequential training.*

③ **Our tailored methods outperform the naïve extensions.** We now evaluate our proposed set of methods against the naïve extensions. For each pair of methods (e.g., Naïve-CL-VREX versus ★-CL-VREX), we compute the performance improvement from our methods. The results are in Figure 4 and Table 7. We observe the following patterns: (1) Our proposed methods consistently improve over the naïve extensions. (2) Across all datasets, the improvement is  $\sim 10$ pp or higher. (3) The performance gains are strongest for RotatedMNIST and real-world datasets such as Coverttype. Our explanation is that, on RotatedMNIST, the naïve methods focus on the relative (but spurious) position of the pixels, which breaks in the target domain. Coverttype introduces a severe domain shift, as an entirely new wilderness area with distinct vegetation patterns and elevation is used as the target domain. Here, our methods can better learn the factors underlying the presence of vegetation and is thus robust wrt. domain shifts.  $\Rightarrow$  **Takeaway:** *Our proposed*

methods consistently and considerably improve over the naïve extensions.

**Discussion:** We highlight three main strengths of this paper. (1) We introduce a *deployment-centric* view on CL, where sequentially trained models are evaluated on an unseen target domain to explicitly assess generalization beyond the training distributions. We encourage future work to adopt this evaluation protocol, as generalization after deployment is a critical yet underexplored aspect of continual learning. (2) We perform a rigorous empirical evaluation, which spans diverse benchmark and real-world datasets and includes extensive hyperparameter tuning, amounting to more than 10’000 individual runs (with full carbon offsetting). (3) We derive a new class of methods for continually learning domain-invariant representations, achieving SOTA performance in generalization to unseen target domains. Overall, our results suggest that ★-CL methods successfully preserve domain-invariant structure and help avoid shortcut learning in sequential settings.

## Impact statement

We expect our findings to be of practical relevance for domains where generalization post-deployment is essential, but where source data cannot be freely shared (as in medicine) or fully stored (as in manufacturing), and where models must thus be trained sequentially. In such scenarios, standard continual learning methods may rely on spurious, domain-specific cues that are predictive during training but fail after deployment—thus suffering from shortcut learning (Geirhos et al., 2020). Our results suggest that continually learning domain-invariant representations can mitigate this risk by discouraging reliance on such shortcuts and instead capturing more stable, generalizable structure. This makes our approach broadly applicable to domains such as computer vision, robotics, manufacturing, and medicine. In medical settings in particular, improved robustness to distribution shifts could benefit patients, although we emphasize the need for cautious validation and responsible deployment.

**Limitations:** We discuss solutions to two limitations of our framework. First, we utilize replay buffers, a common technique in continual learning (e.g., Aljundi et al., 2019b; Buzzega et al., 2020; Sarfraz et al., 2025), for storing domain-specific data. In the domain-incremental source setting that we studied, the domain labels are always considered to be available to maintain the domain partitioning. These labels might not be available under all scenarios. For these settings, we suggest using artificially constructed domain boundaries: while Equations (5) to (7) require domain labels, these labels can also be constructed. Second, we use invariance notions that can be computed on a per-domain batch basis. While this covers a vast majority of the DURL principles used in the literature, (e.g., Sun & Saenko, 2016;

Li et al., 2018; Arjovsky et al., 2019; Parascandolo et al., 2020; Krueger et al., 2021; Rame et al., 2022; Krishnamachari et al., 2024), some invariance notions might require a different representation. In such cases, Equation (6) needs to be adapted. Equation (7) needs to be updated to use domain-wise (and not batch-wise) invariance statistics.

## References

- Aljundi, R., Belilovsky, E., Tuytelaars, T., Charlin, L., Caccia, M., Lin, M., and Page-Caccia, L. Online continual learning with maximal interfered retrieval. *Advances in Neural Information Processing Systems*, 32, 2019a.
- Aljundi, R., Lin, M., Goujaud, B., and Bengio, Y. Gradient based sample selection for online continual learning. *Advances in Neural Information Processing Systems*, 32, 2019b.
- Arjovsky, M., Bottou, L., Gulrajani, I., and Lopez-Paz, D. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.
- Bandi, P., Geessink, O., Manson, Q., Van Dijk, M., Balkenhol, M., Hermesen, M., Bejnordi, B. E., Lee, B., Paeng, K., Zhong, A., et al. From detection of individual metastases to classification of lymph node status at the patient level: the Camelyon17 challenge. *Transactions on Medical Imaging*, 38(2):550–560, 2019.
- Benjamin, A. S., Rolnick, D., and Körding, K. P. Measuring and regularizing networks in function space. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=SkMwpiR9Y7>.
- Blackard, J. Coverttype. UCI Machine Learning Repository, 1998. DOI: <https://doi.org/10.24432/C50K5N>.
- Blackard, J. A. and Dean, D. J. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. In *Computers and Electronics in Agriculture*, 1999.
- Boschini, M., Bonicelli, L., Buzzega, P., Porrello, A., and Calderara, S. Class-incremental continual learning into the extended der-verse. *Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5497–5512, 2022.
- Bruno, P., Quarta, A., and Calimeri, F. Continual learning in medicine: A systematic literature review. *Neural Processing Letters*, 57(1):2, 2025.
- Buzzega, P., Boschini, M., Porrello, A., Abati, D., and Calderara, S. Dark experience for general continual learning: a strong, simple baseline. *Advances in Neural Information Processing Systems*, 33:15920–15930, 2020.

- Caccia, L., Aljundi, R., Asadi, N., Tuytelaars, T., Pineau, J., and Belilovsky, E. New insights on reducing abrupt representation change in online continual learning. *arXiv preprint arXiv:2104.05025*, 2021.
- Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. Efficient lifelong learning with a-gem. In *ICLR*, 2019a.
- Chaudhry, A., Rohrbach, M., Elhoseiny, M., Ajanthan, T., Dokania, P. K., Torr, P. H., and Ranzato, M. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019b.
- Churamani, N., Cheong, J., Kalkan, S., and Gunes, H. Towards causal replay for knowledge rehearsal in continual learning. In *AAAI Bridge Program on Continual Causality*, pp. 63–70. PMLR, 2023.
- Courty, N., Flamary, R., Habrard, A., and Rakotomamonjy, A. Joint distribution optimal transportation for domain adaptation. *Advances in Neural Information Processing Systems*, 30, 2017.
- da Costa, P. R. d. O., Akçay, A., Zhang, Y., and Kaymak, U. Remaining useful lifetime prediction via deep domain adaptation. *Reliability Engineering & System Safety*, 195: 106682, 2020.
- De Lange, M. and Tuytelaars, T. Continual prototype evolution: Learning online from non-stationary data streams. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 8250–8259, 2021.
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reinhold, O., and Roth, A. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi: 10.1126/science.aaa9375. URL <https://www.science.org/doi/abs/10.1126/science.aaa9375>.
- Elsayed, M. and Mahmood, A. R. Addressing loss of plasticity and catastrophic forgetting in continual learning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Eskandar, M., Imtiaz, T., Hill, D., Wang, Z., and Dy, J. STAR: Stability-inducing weight perturbation for continual learning. In Yue, Y., Garg, A., Peng, N., Sha, F., and Yu, R. (eds.), *International Conference on Learning Representations (ICLR)*, volume 2025, pp. 49102–49117, 2025.
- Farias, V. and Jozefiak, A. D. Self-normalized resets for plasticity in continual learning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Feng, H., Yang, Z., Chen, H., Pang, T., Du, C., Zhu, M., Chen, W., and Yan, S. Cosda: Continual source-free domain adaptation. *arXiv preprint arXiv:2304.06627*, 2023.
- Ganin, Y. and Lempitsky, V. Unsupervised domain adaptation by backpropagation. In *International Conference on Machine Learning (ICML)*, pp. 1180–1189. PMLR, 2015.
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Ghifary, M., Kleijn, W. B., Zhang, M., and Balduzzi, D. Domain generalization for object recognition with multi-task autoencoders. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2551–2559, 2015.
- Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. A kernel two-sample test. *The Journal of Machine Learning Research*, 13(1):723–773, 2012.
- Gu, Y., Yang, X., Wei, K., and Deng, C. Not just selection, but exploration: Online class-incremental continual learning via dual view consistency. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7442–7451, 2022.
- Gulrajani, I. and Lopez-Paz, D. In search of lost domain generalization. In *International Conference on Learning Representations (ICLR)*, 2021.
- Hardt, M., Recht, B., and Singer, Y. Train faster, generalize better: Stability of stochastic gradient descent. In *International Conference on Machine Learning (ICML)*, pp. 1225–1234. PMLR, 2016.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern recognition*, pp. 770–778, 2016.
- Hendrycks, D. and Dietterich, T. Benchmarking neural network robustness to common corruptions and perturbations. *arXiv preprint arXiv:1903.12261*, 2019.
- Hermann, K. L., Mobahi, H., Fel, T., and Mozer, M. C. On the foundations of shortcut learning. *arXiv preprint arXiv:2310.16228*, 2023.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Hua, Z., Shi, J., and Dumond, P. Domain-invariant feature exploration for intelligent fault diagnosis under unseen

- and time-varying working conditions. *Mechanical Systems and Signal Processing*, 224:112193, 2025.
- Hurtado, J., Salvati, D., Semola, R., Bosio, M., and Lomonaco, V. Continual learning for predictive maintenance: Overview and challenges. *Intelligent Systems with Applications*, 19:200251, 2023.
- Hwang, U., Lee, J., Shin, J., and Yoon, S. Sf(da)2: Source-free domain adaptation through the lens of data augmentation. *arXiv preprint arXiv:2403.10834*, 2024.
- Iscen, A., Zhang, J., Lazebnik, S., and Schmid, C. Memory-efficient incremental learning through feature adaptation. In *European Conference on Computer Vision*, pp. 699–715, 2020.
- Jang, J.-S. R. Mir-wm811k: Dataset for wafer map failure pattern recognition. <http://mirmlab.org/dataset/public/>, 2015.
- Kang, H., Mina, R. J. L., Madjid, S. R. H., Yoon, J., Hasegawa-Johnson, M., Hwang, S. J., and Yoo, C. D. Forget-free continual learning with winning subnetworks. In *International Conference on Machine Learning (ICML)*, pp. 10734–10750. PMLR, 2022.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Kiyasseh, D., Zhu, T., and Clifton, D. A clinical deep learning framework for continually learning from cardiac signals across diseases, time, modalities, and institutions. *Nature Communications*, 12(1):4221, 2021.
- Koh, P. W., Sagawa, S., Marklund, H., Xie, S. M., Zhang, M., Balsubramani, A., Hu, W., Yasunaga, M., Phillips, R. L., Gao, I., et al. Wilds: A benchmark of in-the-wild distribution shifts. In *International Conference on Machine Learning (ICML)*, pp. 5637–5664. PMLR, 2021.
- Krishnamachari, K., Ng, S.-K., and Foo, C.-S. Uniformly distributed feature representations for fair and robust learning. *Transactions on Machine Learning Research*, 2024.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, 2009.
- Krueger, D., Caballero, E., Jacobsen, J.-H., Zhang, A., Binas, J., Zhang, D., Le Priol, R., and Courville, A. Out-of-distribution generalization via risk extrapolation (rex). In *International Conference on Machine Learning (ICML)*, pp. 5815–5826. PMLR, 2021.
- Le, Y. and Yang, X. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.
- LeCun, Y. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>, 1998.
- Lee, C. S. and Lee, A. Y. Clinical applications of continual learning machine learning. *The Lancet Digital Health*, 2(6):e279–e281, 2020.
- Li, H., Pan, S. J., Wang, S., and Kot, A. C. Domain generalization with adversarial feature learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5400–5409, 2018.
- Li, Y., Wang, N., Shi, J., Liu, J., and Hou, X. Revisiting batch normalization for practical domain adaptation. *arXiv preprint arXiv:1603.04779*, 2016.
- Liang, J., Hu, D., and Feng, J. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International Conference on Machine Learning (ICML) (ICML)*, pp. 6028–6039, 2020.
- Liang, J., Hu, D., Wang, Y., He, R., and Feng, J. Source data-absent unsupervised domain adaptation through hypothesis transfer and labeling transfer. *Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2021.
- Liang, Y.-S. and Li, W.-J. Loss decoupling for task-agnostic continual learning. *Advances in Neural Information Processing Systems*, 36:11151–11167, 2023.
- Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. *Advances in Neural Information Processing Systems*, 30, 2017.
- Lu, A., Yuan, H., Feng, T., and Sun, Y. Rethinking the stability-plasticity trade-off in continual learning from an architectural perspective. *arXiv preprint arXiv:2506.03951*, 2025.
- Magistri, S., Trinci, T., Soutif-Cormerais, A., van de Weijer, J., and Bagdanov, A. D. Elastic feature consolidation for cold start exemplar-free incremental learning. *arXiv preprint arXiv:2402.03917*, 2024.
- Mahajan, D., Tople, S., and Sharma, A. Domain generalization using causal matching. In *International Conference on Machine Learning (ICML)*, pp. 7313–7324. PMLR, 2021.
- Mallya, A. and Lazebnik, S. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7765–7773, 2018.

- McCloskey, M. and Cohen, N. J. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- McInnes, L., Healy, J., and Melville, J. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- Parascandolo, G., Neitz, A., Orvieto, A., Gresele, L., and Schölkopf, B. Learning explanations that are hard to vary. *arXiv preprint arXiv:2009.00329*, 2020.
- Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.
- Qin, C., You, H., Wang, L., Kuo, C.-C. J., and Fu, Y. Pointdan: A multi-scale 3d domain adaption network for point cloud representation. *Advances in Neural Information Processing Systems*, 32, 2019.
- Rame, A., Dancette, C., and Cord, M. Fishr: Invariant gradient variances for out-of-distribution generalization. In *International Conference on Machine Learning (ICML)*, pp. 18347–18377. PMLR, 2022.
- Ratcliff, R. Connectionist models of recognition memory: constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2):285, 1990.
- Rojas-Carulla, M., Schölkopf, B., Turner, R., and Peters, J. Invariant models for causal transfer learning. *Journal of Machine Learning Research*, 19(36):1–34, 2018.
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hassel, R. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Saha, G., Garg, I., and Roy, K. Gradient projection memory for continual learning. *arXiv preprint arXiv:2103.09762*, 2021.
- Sarfraz, F., Arani, E., and Zonooz, B. Sparse coding in a dual memory system for lifelong learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 9714–9722, 2023.
- Sarfraz, F., Arani, E., and Zonooz, B. Semantic aware representation learning for lifelong learning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Serra, J., Suris, D., Miron, M., and Karatzoglou, A. Overcoming catastrophic forgetting with hard attention to the task. In *International Conference on Machine Learning (ICML)*, pp. 4548–4557. PMLR, 2018.
- Shaheen, K., Hanif, M. A., Hasan, O., and Shafique, M. Continual learning for real-world autonomous systems: Algorithms, challenges and frameworks. *Journal of Intelligent & Robotic Systems*, 105(1):9, 2022.
- Shankar, V., Dave, A., Roelofs, R., Ramanan, D., Recht, B., and Schmidt, L. Do image classifiers generalize across time? In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9661–9669, 2021.
- Shi, H. and Wang, H. A unified approach to domain incremental learning with memory: Theory and algorithm. *Advances in Neural Information Processing Systems*, 36: 15027–15059, 2023.
- Shi, Y., Seely, J., Torr, P. H., Hannun, A., Usunier, N., and Synnaeve, G. Gradient matching for domain generalization. *arXiv preprint arXiv:2104.09937*, 2021.
- Sójka, D., Cygert, S., Twardowski, B., and Trzcíński, T. Ar-tta: A simple method for real-world continual test-time adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pp. 3491–3495, October 2023.
- Sugiyama, M., Nakajima, S., Kashima, H., Buenau, P., and Kawanabe, M. Direct importance estimation with model selection and its application to covariate shift adaptation. *Advances in Neural Information Processing Systems*, 20, 2007.
- Sun, B. and Saenko, K. Deep coral: Correlation alignment for deep domain adaptation. In *European Conference on Computer Vision*, pp. 443–450. Springer, 2016.
- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning (ICML)*, pp. 9229–9248. PMLR, 2020.
- Tang, J., Lu, H., Xu, X., Wu, R., Hu, S., Zhang, T., Cheng, T. W., Ge, M., Chen, Y.-C., and Tsung, F. An incremental unified framework for small defect inspection. In *European Conference on Computer Vision*, pp. 307–324, 2024.
- Tanwani, A. Dirl: Domain-invariant representation learning for sim-to-real transfer. In *Conference on Robot Learning*, pp. 1558–1571. PMLR, 2021.
- Thrun, S. and Mitchell, T. M. Lifelong robot learning. *Robotics and autonomous systems*, 15(1-2):25–46, 1995.
- Toldo, M. and Ozay, M. Bring evanescent representations to life in lifelong class incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16732–16741, 2022.

- Van de Ven, G. M., Tuytelaars, T., and Tolias, A. S. Three types of incremental learning. *Nature Machine Intelligence*, 4(12):1185–1197, 2022.
- Vokinger, K. N., Feuerriegel, S., and Kesselheim, A. S. Continual learning in medical devices: Fda’s action plan and beyond. *The Lancet Digital Health*, 3(6):e337–e338, 2021.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*, 2021.
- Wang, L., Ding, Z., and Fu, Y. Low-rank transfer human motion segmentation. *Transactions on Image Processing*, 28(2):1023–1034, 2019.
- Wang, L., Zhang, X., Su, H., and Zhu, J. A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(8):5362–5383, 2024.
- Wang, Q., Fink, O., Van Gool, L., and Dai, D. Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7201–7211, 2022.
- Wang, T., Guo, D., and Sun, X.-M. Contrastive generative replay method of remaining useful life prediction for rolling bearings. *IEEE Sensors Journal*, 23(19):23893–23902, 2023.
- Welford, B. P. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.
- Wu, M.-J., Jang, J.-S. R., and Chen, J.-L. Wafer map failure pattern recognition and similarity ranking for large-scale data sets. *IEEE Transactions on Semiconductor Manufacturing*, 28(1):1–12, 2015. doi: 10.1109/TSM.2014.2364237.
- Yoon, J., Yang, E., Lee, J., and Hwang, S. J. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*, 2017.
- Yoon, J., Kim, S., Yang, E., and Hwang, S. J. Scalable and order-robust continual learning with additive parameter decomposition. *arXiv preprint arXiv:1902.09432*, 2019.
- Zenke, F., Poole, B., and Ganguli, S. Continual learning through synaptic intelligence. In *International Conference on Machine Learning (ICML)*, pp. 3987–3995. PMLR, 2017.
- Zhang, K., Schölkopf, B., Muandet, K., and Wang, Z. Domain adaptation under target and conditional shift. In *International Conference on Machine Learning (ICML)*, pp. 819–827. PMLR, 2013.
- Zhou, K., Liu, Z., Qiao, Y., Xiang, T., and Loy, C. C. Domain generalization: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4):4396–4415, 2022.

## A. Extended Related Works

**Continual learning** (CL) focuses on sequentially training on a stream of data distributions while preventing forgetting (McCloskey & Cohen, 1989; Lu et al., 2025). In the literature, **three scenarios of CL** exist (Van de Ven et al., 2022): (1) Task-incremental learning, where each domain introduces a new task that needs to be learned additionally to the previous tasks. Due to the usage of task-identities during inference, which are not available for *novel domains*, this setting is essentially solved (Mallya & Lazebnik, 2018; Kang et al., 2022). (2) Class-incremental learning, where each domain introduces new classes that need to be learned in addition to (now unavailable) existing classes (Boschini et al., 2022; Gu et al., 2022). (3) Domain-incremental learning, where the number of classes is fixed and shared across all domains, but each domain contains data from a different distribution (Shi & Wang, 2023; Wang et al., 2024). Our *training* is similar to domain-incremental learning, but then, we evaluate on an unseen target domain.

CL methods are commonly grouped into four major categories: (1) Optimization-based methods, which modify gradients to prevent interference with previously learned tasks (Saha et al., 2021) or prevent weight updates (Elsayed & Mahmood, 2024). (2) Regularization based methods, which reduce forgetting by regularizing updates to model weights or features (Kirkpatrick et al., 2017; Zenke et al., 2017; Magistri et al., 2024). (3) Architecture-based methods, which add new network modules for new domains (and hence often scale quadratically or even exponentially with the number of domains) (Rusu et al., 2016; Yoon et al., 2017; 2019). (4) Replay-based methods, which maintain a buffer to replay (i) samples (Chaudhry et al., 2019b; Churamani et al., 2023; Eskandar et al., 2025), (ii) features (Isken et al., 2020; Toldo & Ozay, 2022), or (iii) latent class prototypes (De Lange & Tuytelaars, 2021; Sarfraz et al., 2023; 2025).

**Domain invariant representation learning** (DIRL) methods aim to learn features that are invariant to the domain and thus generalize better to novel domains (Li et al., 2018; Krishnamachari et al., 2024). Such methods are often motivated by the idea that there underlying causal mechanism exists across all domains and that causal mechanism which can be uncovered through joint training (Arjovsky et al., 2019; Krueger et al., 2021; Mahajan et al., 2021; Rame et al., 2022). DIRL methods can be grouped based on the invariance property they target during training: (1) **Features**, where features are aligned across different feature distributions (ie., domains) (Sun & Saenko, 2016; Li et al., 2018; Krishnamachari et al., 2024). (2) **Gradients**, where gradient-statistics are aligned across domains (Parascandolo et al., 2020; Shi et al., 2021; Rame et al., 2022). (3) **Weights**, where a fixed set of weights, such as a classification head, is aligned across domain-specific representations (Rojas-Carulla et al., 2018; Arjovsky et al., 2019). These methods are predominantly used in a multi-source setting, where the training data is divided into a set of multiple distinct source domains (Gulrajani & Lopez-Paz, 2021; Zhou et al., 2022). After *jointly* training on the set of source domains, the model is evaluated on a unseen target domain. Our work is orthogonal to existing DIRL works in that we focus on sequentially learning invariant representation across a *sequence* of source domains, with the goal of improved generalization to a new target domain.

**Domain adaptation** (DA) methods adapt a model from a source to a target domain and require access to data of both domains (Hwang et al., 2024). Commonly, DA is studied under the **unsupervised domain adaptation** (UDA) setting, where access to unlabeled target data is available (Ganin & Lempitsky, 2015; Qin et al., 2019). To bridge the differences between source and target distributions, UDA methods try to match the (i) marginal (Sugiyama et al., 2007; Wang et al., 2019) or (ii) conditional distributions (Zhang et al., 2013; Courty et al., 2017). Some of the methods also lend themselves naturally to **Continual domain adaptation**, where a pre-trained model is continually adapted to a sequence of changing target distributions (Liang et al., 2020; 2021; Feng et al., 2023). Both DA and CDA differ from the setting studied in this paper. In our deployment-oriented evaluation protocol, we perform training on a **supervised** sequence of source domains, and afterwards test on a single unseen target domain. Nonetheless, we later also compare against widely-used DA baseline SHOT++ (Liang et al., 2021).

**Continual test-time adaptation** (CTTA) studies on-the-fly model adaptations at test time to a non-stationary unlabeled (target) stream (Wang et al., 2022). For adaptation, methods use test-time training (Sun et al., 2020), entropy-minimization (Wang et al., 2021), or recomputing normalization statistics (Li et al., 2016). Representative methods include extending test-time augmentations with stochastic weight resetting and EMA teacher-student frameworks (Wang et al., 2022), self-training with a small replay-buffer and dynamic normalization (Sójka et al., 2023). CTTA is different from our setting. We have a sequence of **source** domains and evaluate on a single, fixed target domain. Nonetheless, we later include two widely-used CTTA baselines, namely, TENT (Wang et al., 2021) and CoTTA (Wang et al., 2022).

## B. Details for the naïve domain-invariant CL algorithms

We instantiate Eq. (2) by specifying a method-dependent statistic  $\hat{\phi}_s(\theta, \omega; B_s)$  computed on a minibatch  $B_s \sim D_s$  from the current domain, and a domain prior  $\Phi_{s'}$  computed at the end of training on each past domain  $s' < s$ . All priors are computed via Welford aggregation (Eq. 3) and are fixed once stored. During training on domain  $s$ , the naïve penalty matches the current statistic to past priors (Eq. 4).

**NaïveVREX.** VREx (Krueger et al., 2021) encourages invariance by matching risks across environments. In the naïve sequential variant, we match the current-domain empirical risk to stored domain-level risk summaries.

- **Per-batch statistic (current domain):**

$$\hat{\phi}_s \equiv \hat{r}_s = \mathbb{E}_{(\mathbf{x}, y) \sim B_s} [\mathcal{L}(h(\mathbf{x}; \theta, \omega), y)]. \quad (9)$$

- **Stored prior (past domains):**

$$\Phi_{s'} \equiv \bar{r}_{s'} = \text{AGGREGATE}_{(\mathbf{x}, y) \sim D_{s'}} (\mathcal{L}(h(\mathbf{x}; \theta, \omega), y)), \quad s' < s. \quad (10)$$

- **Naïve penalty:**

$$\mathcal{P}_s^{\text{NAIVEVREX}} = \frac{1}{s-1} \sum_{s'=1}^{s-1} (\hat{r}_s - \bar{r}_{s'})^2. \quad (11)$$

**NaïveFishr.** Fishr (Rame et al., 2022) matches gradient-variance statistics across domains to promote invariance. In the naïve sequential variant, we compute a gradient-variance vector on the current minibatch and match it to stored per-domain prototypes.

Let  $\nabla_{\omega} \mathcal{L}(h(\mathbf{x}; \theta, \omega), y)$  denote the gradient of the loss w.r.t. classifier parameters  $\omega$ .<sup>3</sup>

- **Per-batch statistic (current domain):**

$$\hat{\phi}_s \equiv \hat{\mathbf{v}}_s = \text{Var}_{(\mathbf{x}, y) \sim B_s} (\nabla_{\omega} \mathcal{L}(h(\mathbf{x}; \theta, \omega), y)). \quad (12)$$

- **Stored prior (past domains):**

$$\Phi_{s'} \equiv \bar{\mathbf{v}}_{s'} = \text{AGGREGATE}_{(\mathbf{x}, y) \sim D_{s'}} (\text{Var}(\nabla_{\omega} \mathcal{L}(h(\mathbf{x}; \theta, \omega), y))), \quad s' < s. \quad (13)$$

- **Naïve penalty:**

$$\mathcal{P}_s^{\text{NAIVE-FISHR}} = \frac{1}{s-1} \sum_{s'=1}^{s-1} \|\hat{\mathbf{v}}_s - \bar{\mathbf{v}}_{s'}\|_2^2. \quad (14)$$

**NaïveCORAL.** Deep CORAL (Sun & Saenko, 2016) aligns second-order feature statistics across domains. In the naïve sequential variant, we compute the current feature mean/covariance on a minibatch and match to stored domain-level feature moments.

Let  $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathcal{H}$  be the latent representation of  $\mathbf{x}$ .

- **Per-batch statistic (current domain):**

$$\hat{\phi}_s \equiv (\hat{\boldsymbol{\mu}}_s, \hat{\boldsymbol{\Sigma}}_s), \quad (15)$$

$$\hat{\boldsymbol{\mu}}_s = \mathbb{E}_{\mathbf{x} \sim B_s} [f_{\theta}(\mathbf{x})], \quad (16)$$

$$\hat{\boldsymbol{\Sigma}}_s = \text{COV}_{\mathbf{x} \sim B_s} (f_{\theta}(\mathbf{x})). \quad (17)$$

<sup>3</sup>(1) As in Fishr, the statistic is typically computed layer-wise and concatenated; for brevity we write a single vector. (2) In our work, for computational reasons, we only align gradients of  $\omega$ . (Rame et al. (2022) also found that the difference between aligning the full weights versus just  $\omega$  is minor.)

- **Stored prior (past domains):**

$$\Phi_{s'} \equiv (\bar{\mu}_{s'}, \bar{\Sigma}_{s'}), \quad (\bar{\mu}_{s'}, \bar{\Sigma}_{s'}) = \text{AGGREGATE}_{\mathbf{x} \sim D_{s'}}(f_{\theta}(\mathbf{x}), \text{Cov}(f_{\theta}(\mathbf{x}))). \quad (18)$$

- **Naïve penalty:**

$$\mathcal{P}_s^{\text{NAIVE-CORAL}} = \frac{1}{s-1} \sum_{s'=1}^{s-1} \left( \|\hat{\mu}_s - \bar{\mu}_{s'}\|_2^2 + \|\hat{\Sigma}_s - \bar{\Sigma}_{s'}\|_F^2 \right). \quad (19)$$

**Naïve-MMD.** MMD-based alignment (Li et al., 2018) measures discrepancies between feature distributions via a kernel. To avoid quadratic-time estimators, we follow common practice and use random Fourier features (RFF) to obtain an explicit feature map  $z(\cdot)$  approximating an RBF kernel. We then match the (approximate) kernel mean embedding of the current domain to stored embeddings.

- **Per-batch statistic (current domain):**

$$\hat{\phi}_s \equiv \hat{\mu}_s^z = \mathbb{E}_{\mathbf{x} \sim B_s} [z(f_{\theta}(\mathbf{x}))]. \quad (20)$$

- **Stored prior (past domains):**

$$\Phi_{s'} \equiv \bar{\mu}_{s'}^z = \text{AGGREGATE}_{\mathbf{x} \sim D_{s'}}(z(f_{\theta}(\mathbf{x}))), \quad s' < s. \quad (21)$$

- **Naïve penalty:**

$$\mathcal{P}_s^{\text{NAIVE-MMD}} = \frac{1}{s-1} \sum_{s'=1}^{s-1} \|\hat{\mu}_s^z - \bar{\mu}_{s'}^z\|_2^2. \quad (22)$$

**Naïve-ANDMask (cross-domain gradient sign agreement via stored mean gradients).** ANDMask (Parascandolo et al., 2020) updates parameters only where gradients agree in sign across sources. In our naïve sequential variant, we approximate multi-domain sign agreement by storing, for each past domain  $s' < s$ , a *domain-level mean gradient* (estimated online after training on  $D_{s'}$ ) and combining the stored gradients with the current minibatch gradient.

Let  $B_s \sim D_s$  be the current minibatch, and define the current-domain minibatch gradient

$$g_s = \nabla_{\theta, \omega} \mathcal{L}_{\text{ERM}}(B_s; \theta, \omega). \quad (23)$$

- **Stored prior (past domains):** for each past domain  $s' < s$  and each parameter tensor index  $j$ , we maintain a Welford estimate of the *mean gradient* over minibatches from  $D_{s'}$ :

$$\Phi_{s'}^{(j)} \equiv \bar{g}_{s'}^{(j)} = \text{AGGREGATE}_{B \sim D_{s'}}(\nabla_{\theta, \omega}^{(j)} \mathcal{L}_{\text{ERM}}(B; \theta, \omega)). \quad (24)$$

- **Per-batch statistic (current domain):** the current minibatch gradient tensor  $g_s^{(j)}$  (Eq. 23) for each parameter index  $j$ .
- **Mask construction (hard agreement):** for each parameter index  $j$ , form the set of gradients  $\{\bar{g}_{s'}^{(j)}\}_{s' < s} \cup \{g_s^{(j)}\}$ , compute per-entry sign agreement, and threshold by  $\tau \in [0, 1]$ :

$$\mathbf{m}_s^{(j)} = \mathbb{I} \left( \left| \frac{1}{s} \left( \text{sign}(g_s^{(j)}) + \sum_{s'=1}^{s-1} \text{sign}(\bar{g}_{s'}^{(j)}) \right) \right| \geq \tau \right). \quad (25)$$

- **Masked update rule:** let  $\tilde{g}_s^{(j)} = \frac{1}{s} (g_s^{(j)} + \sum_{s'=1}^{s-1} \bar{g}_{s'}^{(j)})$  be the elementwise average gradient across current and stored past domains. The applied gradient is

$$\nabla_{\theta, \omega}^{(j)} \leftarrow \frac{\mathbf{m}_s^{(j)} \odot \tilde{g}_s^{(j)}}{\frac{1}{|\theta, \omega|^{(j)}} \sum \mathbf{m}_s^{(j)} + \varepsilon}, \quad \varepsilon > 0, \quad (26)$$

where the denominator matches the implementation’s normalization by the mask density (to stabilize update magnitude).

Unlike the other naïve methods, NaïveANDMask is realized as a masked gradient update rather than an explicit additive penalty  $\mathcal{P}_s$ . Conceptually, the procedure enforces invariance by *restricting updates* to coordinates that exhibit consistent gradient signs across domains, approximated here using stored mean gradients.

## C. Details for our CL methods for domain-invariant representations

The naïve template in Equation (2) enforces invariance through *static* priors  $\{\Phi_{s'}\}_{e < s}$ . It can thus not faithfully emulate multi-domain objectives which require *simultaneous* access to multiple domains. To address this, we propose a set of methods which (i) reintroduce multi-domain computation through a memory buffer and (ii) stabilize invariance learning through sequential invariance alignment. The main details are in Section 5; we here provide more details about the individual methods.

**CL-VREX.** The underlying VREX aims to match risks across domains by penalizing the variance of domain-specific risks (Krueger et al., 2021). Using the replay buffer, we compute the per-domain minibatch risks  $\hat{r}_{s'}(B_{s'})$  for all  $s' \leq s$  and define:

$$\mathcal{P}_s^{\text{replay}} = \text{Var}\left(\{\hat{r}_{s'}(B_{s'})\}_{e \leq s}\right). \quad (27)$$

**CL-Fishr.** Fishr matches gradient-variance statistics across domains (Rame et al., 2022). For our sequential setting, we compute per-domain gradient-variance summaries on classifier parameters, denoted by  $\hat{\mathbf{v}}_{s'}(B_{s'})$ , and enforce invariance through variance matching:

$$\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} \left\| \hat{\mathbf{v}}_{s'}(B_{s'}) - \frac{1}{s} \sum_{j \leq s} \hat{\mathbf{v}}_j(B_j) \right\|_2^2. \quad (28)$$

Further, we align replay-domain gradient-variance statistics to stored references  $\Phi_{s'}$  using a scale-invariant representation (log-variance) with

$$\Phi_{s'} = \text{Normalize}(\log(\mathbf{v}_{s'} + \varepsilon)), \quad \mathcal{L}_{\text{align}} = \frac{1}{s-1} \sum_{e < s} \left(1 - \cos\left(\text{Normalize}(\log(\hat{\mathbf{v}}_{s'}(B_{s'}) + \varepsilon)), \Phi_{s'}\right)\right). \quad (29)$$

where the cosine alignment emphasizes *directional* agreement of invariance statistics while being robust to scale changes over training.

**CL-CORAL.** The CORAL methods matches second-order moments of feature, obtained via the feature extractor  $f_\theta$ , across domains (Sun & Saenko, 2016). Using the replay buffer, we compute per-domain feature mean and covariance on minibatches:

$$\hat{\mu}_{s'}(B_{s'}) = \mathbb{E}_{\mathbf{x} \sim B_{s'}}[f_\theta(\mathbf{x})], \quad \hat{\Sigma}_{s'}(B_{s'}) = \text{Cov}_{\mathbf{x} \sim B_{s'}}[f_\theta(\mathbf{x})]. \quad (30)$$

We then enforce invariance by matching these moments across the set of domains in the current training step:

$$\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} \left( \|\hat{\mu}_{s'}(B_{s'}) - \bar{\mu}\|_2^2 + \|\hat{\Sigma}_{s'}(B_{s'}) - \bar{\Sigma}\|_F^2 \right). \quad (31)$$

where  $(\bar{\mu}, \bar{\Sigma})$  denote averages across  $\{e \leq s\}$ . For CL-CORAL, the sequential invariance alignment stores domain-specific references  $\Phi_{s'} = (\mu_{s'}, \Sigma_{s'})$  at the end of each domain and constrains replay-domain statistics as

$$\mathcal{L}_{\text{align}} = \frac{1}{s-1} \sum_{e < s} \left( \|\hat{\mu}_{s'}(B_{s'}) - \mu_{s'}\|_2^2 + \|\hat{\Sigma}_{s'}(B_{s'}) - \Sigma_{s'}\|_F^2 \right). \quad (32)$$

**CL-MMD.** MMD matches feature distributions by minimizing a kernel distance (Gretton et al., 2012). We approximate this using a random Fourier feature (RFF) to obtain a differentiable, minibatch-estimable mean embedding as

$$\mathbf{z}(\mathbf{x}) = \text{RFF}(f_\theta(\mathbf{x})), \quad \hat{\mu}_{s'}^z(B_{s'}) = \mathbb{E}_{\mathbf{x} \sim B_{s'}}[\mathbf{z}(\mathbf{x})]. \quad (33)$$

Using the replay buffer, we then enforce invariance by matching the mean embeddings across domains in the update:

$$\mathcal{P}_s^{\text{replay}} = \frac{1}{s} \sum_{s' \leq s} \left\| \hat{\mu}_{s'}^z(B_{s'}) - \frac{1}{s} \sum_{j \leq s} \hat{\mu}_j^z(B_j) \right\|_2^2. \quad (34)$$

For CL-MMD, the sequential invariance alignment stores domain-level reference embeddings  $\Phi_{s'} = \mu_{s'}^z$  and regularizes replay drift:

$$\mathcal{L}_{\text{align}} = \frac{1}{s-1} \sum_{e < s} \|\hat{\mu}_{s'}^z(B_{s'}) - \mu_{s'}^z\|_2^2. \quad (35)$$

**Seq-ANDMask.** The ANDMask method aims to update parameters using gradient components that agree in sign across domains (Parascandolo et al., 2020). Using the replay buffer, we treat the current domain minibatch and replayed domain minibatches as a set of environments  $\{B_{s'}\}_{e \leq s}$ . For each network parameter, we compute gradients  $\{\nabla_{\theta, \omega} \mathcal{L}_{s'}\}_{e \leq s}$  and form a (soft) agreement mask  $\mathbf{m}$ :

$$\mathbf{m} = \sigma \left( \frac{\left| \frac{1}{s} \sum_{s' \leq s} \text{sign}(\mathbf{g}_{s'}) \right| - \tau}{T} \right), \quad \mathbf{g}_{s'} = \nabla_{\theta, \omega} \mathcal{L}_{\text{ERM}}(B_{s'}), \quad (36)$$

where  $\tau$  is a sparsity threshold,  $T$  is a temperature, and  $\sigma(\cdot)$  is the sigmoid function. The masked update uses the averaged gradient restricted to agreement directions:

$$\tilde{\mathbf{g}} = \frac{\mathbf{m} \odot \left( \frac{1}{s} \sum_{s' \leq s} \mathbf{g}_{s'} \right)}{\text{mean}(\mathbf{m}) + \varepsilon}. \quad (37)$$

To align across domains, we combine ANDMask with logit anchoring and treat the KD gradient as an additional signal whose *conflicting* component is removed before aggregation (projecting onto the non-conflicting subspace). We further adapt  $\tau$  online to avoid degenerate masks (all-0 or all-1) under limited replay diversity.

## D. Pseudocode

We provide pseudocode for our experimental setup below. For the implementation, we (heavily) modified the `DomainBed` library (Gulrajani & Lopez-Paz, 2021).

---

**Algorithm 2** Our proposed methods with replay and invariance alignment.

---

**Input:** Sequential source domains  $\{D_1, \dots, D_k\}$ ; target domain  $D_t$  (evaluation only); model  $h = g_\omega \circ f_\theta$ ; classification loss  $\mathcal{L}$ ; steps per domain  $T$ ; batch size  $B$ .

**Input:** Memory buffer  $M = \bigcup_{s' < s} M_{s'}$  storing tuples  $(\mathbf{x}, y, \tilde{\mathbf{z}}, e)$  with domain tag  $s'$ .

**Input:** Method-specific statistic  $\phi(\cdot)$ ; invariance penalty operator  $\text{INVPENALTY}(\cdot)$ .

**Input:** Domain priors  $\{\Phi_{s'}\}$ , running estimator  $\text{AGGREGATE}(\cdot)$  (e.g., Welford), and alignment distance  $d_{\text{ALIGN}}(\cdot, \cdot)$ .

**Input:** Weights  $\lambda \geq 0$  (invariance),  $\beta \geq 0$  (alignment); optimizer  $\text{OPT}$ .

- 1: Initialize  $(\theta, \omega)$ ;  $M \leftarrow \emptyset$ ;  $\Phi \leftarrow \emptyset$ .
- 2: **for**  $s = 1$  to  $k$  **do**
- 3:   **for**  $t = 1$  to  $T$  **do**
- 4:     Sample current minibatch  $B_s \sim D_s$  with  $|B_s| = B$ .
- 5:     Initialize multi-domain batch set  $\mathcal{B} \leftarrow \{(s, B_s)\}$ .
- 6:     **if**  $M \neq \emptyset$  **then**
- 7:       Sample replay minibatches  $\{B_{s'}\}_{e \in \mathcal{E}}$  from buffer partitions  $\{M_e\}$  (e.g., balanced across domains), where  $\mathcal{E} \subseteq \{1, \dots, s-1\}$ .
- 8:        $\mathcal{B} \leftarrow \mathcal{B} \cup \{(e, B_{s'})\}_{e \in \mathcal{E}}$ .
- 9:     **end if**
- 10:    **(1) Replay-augmented ERM:**
- 11:     $\mathcal{L}_{\text{ERM}} \leftarrow \frac{1}{\sum_{(e, B_{s'}) \in \mathcal{B}} |B_{s'}|} \sum_{(e, B_{s'}) \in \mathcal{B}} \sum_{(\mathbf{x}, y, \cdot) \in B_{s'}} \mathcal{L}(h(\mathbf{x}; \theta, \omega), y)$ .
- 12:    **(2) Replay-enabled invariance penalty:**
- 13:    For each  $(e, B_{s'}) \in \mathcal{B}$  compute  $\hat{\phi}_{s'} \leftarrow \phi(B_{s'}; \theta, \omega)$ .
- 14:     $\mathcal{P}_s^{\text{replay}} \leftarrow \text{INVPENALTY}(\{\hat{\phi}_{s'}\}_{(e, B_{s'}) \in \mathcal{B}})$ .
- 15:    **(3) Sequential invariance alignment to domain priors:**
- 16:    **if**  $\Phi \neq \emptyset$  **then**
- 17:       $\mathcal{L}_{\text{align}} \leftarrow \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} d_{\text{ALIGN}}(\hat{\phi}_{s'}, \Phi_{s'})$ .
- 18:    **else**
- 19:       $\mathcal{L}_{\text{align}} \leftarrow 0$ .
- 20:    **end if**
- 21:    Total loss and update:
- 22:     $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{ERM}} + \lambda \mathcal{P}_s^{\text{replay}} + \beta \mathcal{L}_{\text{align}}$ .
- 23:     $(\theta, \omega) \leftarrow \text{OPT}((\theta, \omega), \nabla_{\theta, \omega} \mathcal{L}_{\text{total}})$ .
- 24:    **end for**
- 25:    Compute and store domain prior for  $D_s$ :
- 26:     $\Phi_s \leftarrow \text{AGGREGATE}(\phi(\mathbf{x}, y; \theta, \omega) \text{ for } (\mathbf{x}, y) \sim D_s)$ .
- 27: **end for**
- 28: **Evaluate** final model  $h$  on target domain  $D_t$ .

---

## E. Datasets

We used the following classification datasets. For each datasets, we construct a deployment-centric version by selecting a source-domain sequence and a held-out target domain, on which performance is evaluated.

1. **RotatedMNIST** (Ghifary et al., 2015) applies counter-clockwise rotations to the original MNIST dataset. We use source domains  $D^0, D^{15}, D^{30}, D^{45}$  and target domain  $D^{75}$ , where the superscript indicates the rotation in degree applied to the digits. The dataset has 70 000 black-and-white images in resolution  $(1 \times 28 \times 28)$  which are evenly split into the aforementioned source and target domains. Raw images (i.e., before applying the rotation) are assigned to one domain only, ensuring no (raw-data) overlap across domains. The dataset has 10 classes.
2. **CIFAR10C** (Hendrycks & Dietterich, 2019) applies corruptions to CIFAR10 images. To create source and target domains, we joined separate domains from the corruptions described in (Hendrycks & Dietterich, 2019). The source domains are  $D^{\text{clean}}, D^{\text{gaussian+shot}}, D^{\text{impulse+defocus}}, D^{\text{glass-blurr+motion-blurr}}, D^{\text{zoom-blur+snow}}, D^{\text{frost+fog}}$  and  $D^{\text{brightness+contrast}}$ , where the superscript denotes the corruptions applied to the images (except  $D^{\text{clean}}$  where no corruptions are applied). As target domain, we selected  $D^{\text{elastic-transform+pixelate+jpeg-compression+splat+saturate}}$  because it consists of the most challenging corruptions as mentioned in (Hendrycks & Dietterich, 2019). Additionally, the target-domain corruptions present a different type, namely “digital” corruptions (plus spatter and saturate, which have minimal overlap to any of the source corruptions). We ensured that no (corrupted) image from the target domain is contained in any of the source domains. Each domain, except  $D^{\text{clean}}$  and the target domain, has 16 000 color images in  $(3 \times 32 \times 32)$  resolution.  $D^{\text{clean}}$  contains the 50 000 unmodified images from the CIFAR10 training set (Krizhevsky, 2009). The target domain is joined from the five domains indicated in its superscript and consists of 10 000 images. Each domain has ten classes.
3. **TinyImageNetC** (Hendrycks & Dietterich, 2019) is similar to CIFAR10C except that it applies corruptions to the TinyImageNet dataset (Le & Yang, 2015). The dataset contains  $(3 \times 64 \times 64)$ -sized images from 200 object categories (i.e., classes). As before,  $D^{\text{clean}}$  contains unmodified images from the training dataset (100 000 samples), and the other domains are joint across two (for the source domains) or three (for the target domain, namely elastic – transform + pixelate + jpeg – compression) separate corruption types. The source domains contain 16 000 images each, except  $D^{\text{clean}}$  which contains 100 000 images. The target domain contains 6000 unseen images; as with CIFAR10C, we ensured that no image from the target domain is contained in any of the source domains. This way, we avoided any information leakage – essentially leading to a “double distribution shift” (unseen images and unseen corruption types).
4. **WM811K** (Jang, 2015; Wu et al., 2015) contains 811 457 real-world wafer maps and annotations of common failure types. The image data has pixel values 0, 1, 2, where 0 is background, 1 are OK regions, and 2 are faulty regions on the wafer. Depending on the clustering of the failures, one of eight class labels is assigned. Images without any failures are labelled with an additional class label. We resize all wafer maps to a common  $(48 \times 48)$  resolution, rescale the data to  $(0, 1)$  by dividing by 2, and then normalize using ImageNet statistics. We build independent source and train domains by using the provided production lot information to group wafer maps into domain: four non-overlapping source domains  $D^{(1)}, D^{(2)}, D^{(3)}, D^{(4)}$  and a fifth disjoint target domain  $D^{(5)}$ .
5. **Coverttype** (Blackard & Dean, 1999) contains cartographic and environmental measurements for predicting forest cover type. After preprocessing, the dataset consists of 65 386 instances with 50 features, including continuous attributes (e.g., elevation, slope, distances to hydrology, roads, and fire points) and binary indicators encoding, e.g., soil type. Each sample is labeled with one of six forest cover types. To construct source and target domains, we partitioned the data by wilderness area, which induces natural domain shifts corresponding to distinct geographic regions with different environmental characteristics. The source domains are  $D^{\text{Rawah}}, D^{\text{Neota}}, D^{\text{Comanche}}$ , while the target domain is  $D^{\text{CacheLaPoudre}}$ , where the superscript denotes the wilderness area from which the samples originate. This split ensures that the target domain corresponds to a geographically distinct region not observed during training. The source domains contain approximately 45 000 instances in total, while the target domain consists of approximately 9100 instances. All domains share the same feature space and label set. Within each domain, we balanced the classes by subsampling all classes to the same number, given by the minority class.
6. **Camelyon17** (Bandi et al., 2019; Koh et al., 2021) contains histopathology image patches extracted from whole-slide lymph node sections collected at five different hospitals. Each sample is a  $96 \times 96$  color image and the binary

label indicates whether the central  $32 \times 32$  region contains tumor tissue; the domain corresponds to the hospital of origin. We adapt the split proposed by (Koh et al., 2021): we use a sequence of four hospitals with approximately 340 000 class-balanced samples in total as source domain sequence, and use the fifth hospital as the target domain (approximately 85 000 patches).

## F. Hyperparameter details

We here give details about the hyperparameters and the corresponding search ranges. Several of the hyperparameters are shared across all methods, notably the learning rate, weight decay, and the batch size. The ranges of the hyperparameters of the CL baseline methods are taken from their respective papers. To tune the hyperparameters, we followed common practice in CL and split the data from each *source* domain into a 80 % training set and 20 % validation set. The hyperparameters were then tuned using randomized search with 20 trials across the joint distributions described in Table 4. The best hyperparameters were then selected as those that maximize the performance on the validation sets. This way, no information about the unseen target domain is leaked.

This process was repeated three times, yielding 60 trials in total. The numbers reported in this paper are the mean and standard error across the best runs, with one best run per seed (i.e., across three runs/seeds).

The number of training steps was *not* optimized. All methods thus have the same computational budget, and the number of training steps depends on the dataset: 1000 for RotatedMNIST, 1200 for CIFAR10C, 2500 for TinyImageNetC, 500 for WM811K and UCICoverttype, and 1000 for Camelyon17.

Table 4. Hyperparameters, default values, and random-search distributions. **Uniform**( $a, b$ ) denotes continuous sampling; **UniformInt**( $a, b$ ) denotes integer sampling; **Choice**{ $\cdot$ } denotes discrete sampling. Expressions such as  $10^{\text{Uniform}(a,b)}$  and  $2^{\text{Uniform}(a,b)}$  follow the code. Parameters marked *Fixed* are not tuned (random value equals default).

| Condition / Method                                                     | Hyperparameter (name in code)  | Default              | Random distribution                                                |
|------------------------------------------------------------------------|--------------------------------|----------------------|--------------------------------------------------------------------|
| <b>Dataset-specific (shared across algorithms)</b>                     |                                |                      |                                                                    |
| RotatedMNIST / Coverttype                                              | Replay buffer capacity         | 1000                 | Fixed(1000)                                                        |
| All other datasets                                                     | Replay buffer capacity         | 5000                 | Fixed(5000)                                                        |
| <b>Learning rate / regularization / batch size (dataset-dependent)</b> |                                |                      |                                                                    |
| SMALL_DATASETS <sup>†</sup>                                            | Learning rate                  | $10^{-3}$            | $10^{\text{Uniform}(-4.5, -0.5)}$                                  |
| Other datasets                                                         | Learning rate                  | $5 \cdot 10^{-5}$    | $10^{\text{Uniform}(-5, -0.5)}$                                    |
| SMALL_DATASETS <sup>†</sup>                                            | Weight decay                   | 0                    | Fixed(0)                                                           |
| Other datasets                                                         | Weight decay                   | 0                    | $10^{\text{Uniform}(-6, -2)}$                                      |
| SMALL_DATASETS <sup>†</sup>                                            | Batch size                     | 64                   | $2^{\text{Uniform}(3, 8)}$                                         |
| Other datasets                                                         | Batch size                     | 32                   | $2^{\text{Uniform}(3, 5)}$                                         |
| <b>Algorithm-specific hyperparameters</b>                              |                                |                      |                                                                    |
| Naïve-CL-CORAL                                                         | CORAL penalty weight $\lambda$ | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
| ★-CL-CORAL                                                             | CORAL penalty weight $\lambda$ | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
|                                                                        | Alignment-to-prior weight      | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
| Naïve-CL-MMD                                                           | MMD penalty weight $\lambda$   | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
|                                                                        | #RFF features (mmd_gamma)      | 128                  | Fixed(128)                                                         |
| ★-CL-MMD                                                               | MMD penalty weight $\lambda$   | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
|                                                                        | Alignment-to-prior weight      | 1.0                  | $10^{\text{Uniform}(-1, 1)}$                                       |
|                                                                        | #RFF features                  | 128                  | Fixed(128)                                                         |
| Naïve-CL-VREX                                                          | V-REx penalty weight $\lambda$ | $10^1$               | $10^{\text{Uniform}(-1, 5)}$                                       |
|                                                                        | Penalty anneal iters           | $100 / 500^\ddagger$ | $\text{UniformInt}(0, 1000) / \text{UniformInt}(0, 2500)^\ddagger$ |
| ★-CL-VREX                                                              | V-REx penalty weight $\lambda$ | $10^1$               | $10^{\text{Uniform}(-1, 5)}$                                       |
|                                                                        | Penalty anneal iters           | $100 / 500^\ddagger$ | $\text{UniformInt}(0, 1000) / \text{UniformInt}(0, 2500)^\ddagger$ |
|                                                                        | Alignment weight               | 1.0                  | $\text{Uniform}(1.0, 1000.0)$                                      |
| Naïve-CL-Fishr                                                         | Fishr penalty weight $\lambda$ | 1000                 | $10^{\text{Uniform}(1, 4)}$                                        |
|                                                                        | Penalty anneal iters           | $100 / 500^\ddagger$ | $\text{UniformInt}(0, 1000) / \text{UniformInt}(0, 2500)^\ddagger$ |
| ★-CL-Fishr                                                             | Fishr penalty weight $\lambda$ | 1000                 | $10^{\text{Uniform}(1, 4)}$                                        |
|                                                                        | Penalty anneal iters           | $100 / 500^\ddagger$ | $\text{UniformInt}(0, 1000) / \text{UniformInt}(0, 2500)^\ddagger$ |
|                                                                        | Alignment weight $\beta$       | 1.0                  | $\text{Uniform}(1.0, 1000.0)$                                      |
| Naïve-CL-ANDMASK                                                       | $\tau$                         | 1                    | $\text{Uniform}(0.5, 1.0)$                                         |
| ★-CL-ANDMask                                                           | Agreement threshold $\tau$     | 1                    | $\text{Uniform}(0.5, 1.0)$                                         |
|                                                                        | Alignment weight $\alpha$      | 0.1                  | $\text{Uniform}(0.1, 0.5)$                                         |
| EWC (Kirkpatrick et al., 2017)                                         | Regularization $\lambda$       | 10.0                 | $\text{Choice}\{0.1, 1.0, 5.0, 10.0, 30.0, 90.0, 100.0\}$          |
|                                                                        | Fisher decay $\gamma$          | 0.9                  | $\text{Choice}\{0.8, 0.9, 1.0\}$                                   |
| SI (Zenke et al., 2017)                                                | $\xi$                          | 0.1                  | $\text{Uniform}(0.001, 0.1)$                                       |
|                                                                        | $c$                            | 0.1                  | $\text{Uniform}(0.001, 0.1)$                                       |
| STAR (Eskandar et al., 2025)                                           | $\gamma$                       | 0.01                 | $\text{Uniform}(0.01, 0.05)$                                       |
|                                                                        | $\lambda$                      | 0.01                 | $\text{Uniform}(0.01, 0.05)$                                       |
| LODE (Liang & Li, 2023)                                                | $\rho$                         | 0.1                  | $\text{Choice}\{0.01, 0.05, 0.1, 0.2\}$                            |
| COPE (De Lange & Tuytelaars, 2021)                                     | $\tau$                         | 1                    | $\text{Uniform}(0.5, 1.0)$                                         |
|                                                                        | $\gamma$                       | 0.9                  | $\text{Choice}\{0.8, 0.9, 1.0\}$                                   |
|                                                                        | Inner repeats                  | 3                    | $\text{Choice}\{1, 2, 3, 4, 5\}$                                   |
| SARL (Sarfratz et al., 2025)                                           | Replay logit anchoring         | 1.0                  | $\text{Choice}\{0.2, 0.5, 1.0\}$                                   |
|                                                                        | Stable-model anchoring         | 1.0                  | Fixed(1.0)                                                         |
|                                                                        | Regularization                 | 1.0                  | $\text{Choice}\{0.2, 0.5, 1.0\}$                                   |
|                                                                        | Semantics $\tau$               | 0.8                  | Fixed(0.8)                                                         |
|                                                                        | Semantics weight               | 0.01                 | Fixed(0.01)                                                        |
|                                                                        | Warmup steps                   | $100 / 500^\ddagger$ | $\text{UniformInt}(0, 1000) / \text{UniformInt}(0, 2500)^\ddagger$ |
| EFC                                                                    | $\lambda$                      | 0.4                  | $\text{Uniform}(0.01, 0.5)$                                        |
|                                                                        | $\alpha$                       | 0.5                  | $\text{Uniform}(0.01, 0.99)$                                       |
|                                                                        | $\delta$                       | 0.9                  | $\text{Uniform}(0.001, 0.1)$                                       |

<sup>†</sup>SMALL\_DATASETS = {RotatedMNIST, CIFAR10C, Coverttype}.

<sup>‡</sup>For datasets in {RotatedMNIST, Coverttype, Camelyon17}, the default range is 100 and UniformInt(0, 1000); otherwise 500 and UniformInt(0, 2500)

## G. Extended results

### G.1. Comparison against CDA/CTTA baselines

In Table 5, we provide experimental results for CDA/CTTA baselines TENT (Wang et al., 2021), SHOT++ (Liang et al., 2021), and CoTTA (Wang et al., 2022). We observe that our proposed  $\star$ -CL methods outperform the baselines by a substantial margin.

Table 5. **Extended main results:** Comparison of our proposed methods against CDA and CTTA baselines on six benchmark datasets. For these baselines, we allow unsupervised updates on the target domain. We report the mean $\pm$ standard error target domain performance across three independent runs. Results marked as **best**, second, third.

| Method              | RotatedMNIST                     | CIFAR10C                         | TinyImageNetC                    | WM811K                           | Coverttype                       | Camelyon17                       | Avg         | Arith. mean | Geom. mean | Median     |
|---------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|-------------|-------------|------------|------------|
| Finetune            | 39.5 $\pm$ 1.7                   | 66.3 $\pm$ 0.4                   | 18.5 $\pm$ 2.6                   | 82.8 $\pm$ 0.3                   | 8.1 $\pm$ 0.0                    | 86.9 $\pm$ 2.1                   | 50.4        | 6.7         | 6.6        | 6.5        |
| $\star$ -CL-VREX    | 70.2 $\pm$ 5.8                   | 67.2 $\pm$ 1.4                   | 26.3 $\pm$ 1.2                   | 84.1 $\pm$ 0.3                   | 40.8 $\pm$ 1.5                   | 91.5 $\pm$ 0.6                   | 63.4        | 3.2         | 3.0        | 3.5        |
| $\star$ -CL-Fishr   | 68.6 $\pm$ 0.4                   | 64.2 $\pm$ 1.2                   | <b>29.0 <math>\pm</math> 2.2</b> | 83.6 $\pm$ 0.5                   | 27.0 $\pm$ 7.7                   | 89.7 $\pm$ 2.0                   | 60.3        | 5.0         | 4.3        | 5.0        |
| $\star$ -CL-CORAL   | <b>72.8 <math>\pm</math> 2.6</b> | 68.5 $\pm$ 1.7                   | 25.0 $\pm$ 2.8                   | 84.8 $\pm$ 0.6                   | <b>45.2 <math>\pm</math> 3.9</b> | <b>91.7 <math>\pm</math> 0.4</b> | <b>64.7</b> | <b>1.8</b>  | <b>1.6</b> | <b>1.5</b> |
| $\star$ -CL-MMD     | 70.7 $\pm$ 0.5                   | <b>69.0 <math>\pm</math> 0.5</b> | 25.8 $\pm$ 0.8                   | <b>85.5 <math>\pm</math> 0.4</b> | 37.6 $\pm$ 0.9                   | 90.1 $\pm$ 0.5                   | 63.1        | 2.7         | 2.3        | 3.0        |
| $\star$ -CL-ANDMask | 71.4 $\pm$ 2.6                   | 64.7 $\pm$ 0.7                   | 11.8 $\pm$ 3.6                   | 84.6 $\pm$ 0.6                   | 43.7 $\pm$ 2.9                   | 89.1 $\pm$ 1.5                   | 60.9        | 4.3         | 3.8        | 4.5        |
| TENT                | 28.0 $\pm$ 0.6                   | 67.9 $\pm$ 1.0                   | 20.2 $\pm$ 0.2                   | 77.1 $\pm$ 0.4                   | 36.8 $\pm$ 1.8                   | 91.4 $\pm$ 2.1                   | 53.6        | 5.7         | 5.2        | 5.5        |
| CoTTA               | 29.8 $\pm$ 9.5                   | 62.6 $\pm$ 4.3                   | 7.3 $\pm$ 5.6                    | 78.2 $\pm$ 0.3                   | 36.9 $\pm$ 0.9                   | 82.5 $\pm$ 2.4                   | 49.5        | 7.3         | 7.2        | 8.0        |
| SHOTPP              | 49.8 $\pm$ 10.1                  | 19.1 $\pm$ 1.8                   | 1.0 $\pm$ 0.3                    | 57.5 $\pm$ 0.6                   | 22.3 $\pm$ 9.1                   | 67.8 $\pm$ 6.6                   | 36.2        | 8.3         | 8.2        | 9.0        |

Macro F1 for WM811K, accuracy for other datasets

### G.2. Naïve results

We here give the full tabular results for the naïve extensions.

Table 6. Naïve extensions versus CL baselines. Shown: mean $\pm$ standard error target domain accuracy across three independent runs. Marked: **best**, second, third.

| Method           | RotatedMNIST                     | CIFAR10C                         | TinyImageNetC                    | WM811K                           | Coverttype                       | Camelyon17                       | Avg         | $\Delta$ to ERM |
|------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|-------------|-----------------|
| Finetune         | 39.5 $\pm$ 1.7                   | 66.3 $\pm$ 0.4                   | 18.5 $\pm$ 2.6                   | 82.8 $\pm$ 0.3                   | 8.1 $\pm$ 0.0                    | 86.9 $\pm$ 2.1                   | 50.4        | +0.0            |
| AGEM             | <u>54.8 <math>\pm</math> 4.7</u> | <b>69.2 <math>\pm</math> 0.2</b> | <u>22.8 <math>\pm</math> 1.1</u> | <u>84.1 <math>\pm</math> 0.2</u> | <u>29.0 <math>\pm</math> 8.5</u> | <b>91.0 <math>\pm</math> 0.1</b> | <u>58.5</u> | <u>+8.1</u>     |
| SI               | 38.8 $\pm$ 0.7                   | 64.3 $\pm$ 2.4                   | 20.8 $\pm$ 1.2                   | 83.3 $\pm$ 0.6                   | <u>24.5 <math>\pm</math> 6.8</u> | 86.0 $\pm$ 3.6                   | 52.9        | +2.6            |
| ER-ACE           | <b>67.5 <math>\pm</math> 2.0</b> | 66.9 $\pm$ 1.1                   | <b>29.0 <math>\pm</math> 1.2</b> | <b>85.4 <math>\pm</math> 0.4</b> | <b>37.7 <math>\pm</math> 0.6</b> | <u>90.3 <math>\pm</math> 1.5</u> | <b>62.8</b> | <b>+12.4</b>    |
| Naïve-CL-VREX    | 45.8 $\pm$ 6.8                   | <u>67.4 <math>\pm</math> 0.3</u> | 20.0 $\pm$ 1.3                   | 84.0 $\pm$ 0.1                   | 8.1 $\pm$ 0.1                    | 88.7 $\pm$ 1.5                   | 52.3        | +1.9            |
| Naïve-CL-Fishr   | 41.4 $\pm$ 1.5                   | <u>67.7 <math>\pm</math> 0.3</u> | 22.2 $\pm$ 2.2                   | 83.8 $\pm$ 0.5                   | 8.2 $\pm$ 0.1                    | 84.6 $\pm$ 5.5                   | 51.3        | +0.9            |
| Naïve-CL-CORAL   | <u>50.2 <math>\pm</math> 5.3</u> | 66.3 $\pm$ 0.8                   | <u>23.6 <math>\pm</math> 0.6</u> | 83.6 $\pm$ 0.1                   | 8.0 $\pm$ 0.1                    | 90.1 $\pm$ 0.7                   | <u>53.6</u> | <u>+3.3</u>     |
| Naïve-CL-MMD     | 39.9 $\pm$ 1.0                   | 66.3 $\pm$ 0.8                   | 19.8 $\pm$ 1.0                   | <u>84.3 <math>\pm</math> 0.3</u> | 7.8 $\pm$ 0.3                    | <u>90.7 <math>\pm</math> 1.9</u> | 51.5        | +1.1            |
| Naïve-CL-ANDMask | 21.5 $\pm$ 4.5                   | 26.9 $\pm$ 1.4                   | 0.5 $\pm$ 0.1                    | 8.7 $\pm$ 0.9                    | 6.6 $\pm$ 1.2                    | 49.5 $\pm$ 1.0                   | 19.0        | -31.4           |

Macro F1 for WM811K, accuracy for other datasets

### G.3. Naïve versus tailored methods

We here give the full tabular results for benchmarking our proposed methods against the naïve counterparts.

### G.4. Backwards transfer

We here give results for computing the backwards transfer metric (BWT) (Lopez-Paz & Ranzato, 2017). BWT measures the performance loss by contrasting the original performance on a source domain with the performance after training. For BWT, negative numbers imply forgetting, while positive numbers indicate a retrospective accuracy improvement (which is desired). The BWT results are in Table 8.

### G.5. Runtimes

We here provide runtimes. We provide the total average runtimes in Table 9, and a comparison of per-step runtimes in Figures 5 and 6. We find that both total and per-step runtimes are comparable to or lower than existing works. To further validate this, we performed additional experiments with extended versions of datasets RotatedMNIST, WM811K, and

Table 7. Our methods against the naïve baselines. Our methods considerably outperform their naïve counterparts. Shown: mean±standard error target domain performance across three independent runs, using macro F1 for WM811K and accuracy for the other datasets. Marked: **best**, second, third.

| Method           | RotatedMNIST      | CIFAR10C          | TinyImageNetC     | WM811K            | Coverttype        | Camelyon17        | Avg         |
|------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------|
| Finetune         | 39.5 ± 1.7        | 66.3 ± 0.4        | 18.5 ± 2.6        | 82.8 ± 0.3        | 8.1 ± 0.0         | 86.9 ± 2.1        | 50.4        |
| Naïve-CL-VREX    | 45.8 ± 6.8        | 67.4 ± 0.3        | 20.0 ± 1.3        | 84.0 ± 0.1        | 8.1 ± 0.1         | 88.7 ± 1.5        | 52.3        |
| Naïve-CL-Fishr   | 41.4 ± 1.5        | 67.7 ± 0.3        | 22.2 ± 2.2        | 83.8 ± 0.5        | 8.2 ± 0.1         | 84.6 ± 5.5        | 51.3        |
| Naïve-CL-CORAL   | 50.2 ± 5.3        | 66.3 ± 0.8        | 23.6 ± 0.6        | 83.6 ± 0.1        | 8.0 ± 0.1         | 90.1 ± 0.7        | 53.6        |
| Naïve-CL-MMD     | 39.9 ± 1.0        | 66.3 ± 0.8        | 19.8 ± 1.0        | 84.3 ± 0.3        | 7.8 ± 0.3         | 90.7 ± 1.9        | 51.5        |
| Naïve-CL-ANDMask | 21.5 ± 4.5        | 26.9 ± 1.4        | 0.5 ± 0.1         | 8.7 ± 0.9         | 6.6 ± 1.2         | 49.5 ± 1.0        | 19.0        |
| ★-CL-VREX        | 70.2 ± 5.8        | 67.2 ± 1.4        | 26.3 ± 1.2        | 84.1 ± 0.3        | 40.8 ± 1.5        | 91.5 ± 0.6        | 63.4        |
| ★-CL-Fishr       | 68.6 ± 0.4        | 64.2 ± 1.2        | <b>29.0 ± 2.2</b> | 83.6 ± 0.5        | 27.0 ± 7.7        | 89.7 ± 2.0        | 60.3        |
| ★-CL-CORAL       | <b>72.8 ± 2.6</b> | 68.5 ± 1.7        | 25.0 ± 2.8        | 84.8 ± 0.6        | <b>45.2 ± 3.9</b> | <b>91.7 ± 0.4</b> | <b>64.7</b> |
| ★-CL-MMD         | 70.7 ± 0.5        | <b>69.0 ± 0.5</b> | 25.8 ± 0.8        | <b>85.5 ± 0.4</b> | 37.6 ± 0.9        | 90.1 ± 0.5        | 63.1        |
| ★-CL-ANDMask     | 71.4 ± 2.6        | 64.7 ± 0.7        | 11.8 ± 3.6        | 84.6 ± 0.6        | 43.7 ± 2.9        | 89.1 ± 1.5        | 60.9        |

Table 8. Backwards transfer for the main results. Shown: mean±standard error source domain backwards transfer in percentage points across three trials.

| Method       | RotatedMNIST      | CIFAR10C          | TinyImageNetC     | WM811K            | Coverttype        | Camelyon17       | Avg         |
|--------------|-------------------|-------------------|-------------------|-------------------|-------------------|------------------|-------------|
| Finetune     | 7.5 ± 4.7         | 11.4 ± 3.0        | 25.5 ± 1.8        | 5.1 ± 0.8         | 11.8 ± 3.1        | 6.5 ± 2.7        | 11.3        |
| AGEM         | 4.5 ± 3.5         | 8.2 ± 1.9         | 9.9 ± 8.1         | 5.8 ± 1.1         | 11.7 ± 2.3        | 5.3 ± 1.5        | 7.6         |
| UPGD         | 13.9 ± 2.2        | 9.0 ± 1.6         | 17.7 ± 6.1        | <b>13.1 ± 1.0</b> | -3.7 ± 2.4        | 3.8 ± 1.4        | 9.0         |
| EWC          | 7.3 ± 4.1         | 12.2 ± 2.5        | 13.3 ± 11.8       | 4.3 ± 1.0         | <b>21.8 ± 1.1</b> | <b>9.9 ± 0.7</b> | <b>11.5</b> |
| SI           | 7.5 ± 4.2         | 6.6 ± 1.1         | 6.6 ± 9.6         | 5.2 ± 0.7         | 12.2 ± 5.4        | 9.5 ± 4.0        | 7.9         |
| SNR          | 7.9 ± 4.0         | <b>12.3 ± 2.4</b> | 19.2 ± 13.7       | 8.6 ± 1.8         | 9.8 ± 3.1         | 7.7 ± 5.5        | 10.9        |
| FDR          | 4.3 ± 3.0         | 10.2 ± 1.5        | 8.4 ± 4.9         | 3.5 ± 0.6         | 18.3 ± 2.9        | 2.9 ± 0.2        | 7.9         |
| ER-ACE       | 10.3 ± 4.5        | 7.1 ± 2.3         | 13.9 ± 1.7        | 4.8 ± 0.6         | 18.2 ± 2.1        | 3.0 ± 0.4        | 9.5         |
| LODE         | 8.4 ± 4.7         | 9.7 ± 2.6         | <b>29.9 ± 2.7</b> | 4.0 ± 0.4         | 8.0 ± 5.4         | 8.4 ± 3.4        | 11.4        |
| STAR         | <b>18.4 ± 1.9</b> | 7.4 ± 1.8         | 7.9 ± 1.9         | 5.0 ± 0.2         | 20.9 ± 0.8        | 4.2 ± 0.1        | 10.6        |
| COPE         | 10.4 ± 4.2        | 9.1 ± 1.5         | 5.9 ± 13.7        | 5.6 ± 1.4         | 1.8 ± 1.4         | 7.0 ± 1.7        | 6.6         |
| EFC          | 4.1 ± 2.9         | 10.7 ± 1.8        | 19.6 ± 3.9        | 3.1 ± 0.7         | 11.1 ± 2.3        | 5.6 ± 1.4        | 9.0         |
| SARL         | 0.8 ± 0.5         | 2.5 ± 0.7         | -0.8 ± 0.5        | 4.9 ± 0.7         | 18.9 ± 3.1        | -1.3 ± 1.8       | 4.2         |
| ★-CL-VREX    | 2.9 ± 1.9         | 8.5 ± 2.4         | 15.3 ± 2.3        | 4.6 ± 0.4         | 13.4 ± 3.3        | 6.0 ± 1.7        | 8.4         |
| ★-CL-Fishr   | 10.9 ± 3.9        | 3.4 ± 1.3         | 16.9 ± 1.1        | 2.9 ± 0.5         | 8.0 ± 0.9         | 3.8 ± 0.5        | 7.7         |
| ★-CL-CORAL   | 10.3 ± 4.6        | 7.4 ± 1.8         | 22.7 ± 5.3        | 7.7 ± 1.1         | 12.3 ± 1.7        | 3.5 ± 1.0        | 10.7        |
| ★-CL-MMD     | 11.1 ± 4.3        | 7.1 ± 1.3         | 8.0 ± 1.6         | 6.8 ± 1.4         | 14.1 ± 1.9        | 2.2 ± 0.4        | 8.2         |
| ★-CL-ANDMask | 10.4 ± 4.6        | 8.4 ± 0.9         | 8.7 ± 6.4         | 4.9 ± 0.8         | 12.7 ± 3.0        | 5.2 ± 1.5        | 8.4         |

Camelyon17. For the extended versions, we create 15 source domains by duplicating source domains. (This experiment was solely conducted as a runtime check, not for results-based benchmarking purposes.) The results for the "+Extended" datasets are in Figure 7. We observe that, also on considerably (+200 %) longer domain sequences, the per-step runtimes remain comparable to the baselines.

Table 9. Runtimes for the methods. Shown: mean  $\pm$  standard error runtimes in minutes across three trials.

| Method       | RotatedMNIST                     | CIFAR10C                         | TinyImageNetC                      | WM811K                          | Coverttype                      | Camelyon17                        | Avg         |
|--------------|----------------------------------|----------------------------------|------------------------------------|---------------------------------|---------------------------------|-----------------------------------|-------------|
| Finetune     | 53.7 $\pm$ 3.9                   | 257.6 $\pm$ 20.2                 | 756.9 $\pm$ 8.9                    | 1.5 $\pm$ 0.3                   | 4.9 $\pm$ 0.1                   | <b>133.5 <math>\pm</math> 2.3</b> | 201.3       |
| AGEM         | 57.2 $\pm$ 4.7                   | 406.3 $\pm$ 2.3                  | 789.5 $\pm$ 24.2                   | 1.5 $\pm$ 0.0                   | 4.6 $\pm$ 0.1                   | 204.6 $\pm$ 33.7                  | 244.0       |
| UPGD         | <b>27.8 <math>\pm</math> 1.2</b> | <b>14.3 <math>\pm</math> 1.5</b> | 117.8 $\pm$ 15.2                   | 7.8 $\pm$ 0.5                   | 0.8 $\pm$ 0.0                   | 173.9 $\pm$ 20.9                  | <u>57.1</u> |
| EWC          | 54.8 $\pm$ 3.3                   | 278.5 $\pm$ 25.8                 | 1891.5 $\pm$ 760.9                 | 2.4 $\pm$ 0.0                   | 8.1 $\pm$ 0.1                   | 255.0 $\pm$ 89.2                  | 415.0       |
| SI           | 56.2 $\pm$ 5.8                   | 309.1 $\pm$ 6.4                  | 820.9 $\pm$ 11.4                   | 1.7 $\pm$ 0.3                   | 5.0 $\pm$ 0.4                   | 158.6 $\pm$ 17.8                  | 225.2       |
| SNR          | 44.7 $\pm$ 6.3                   | 22.6 $\pm$ 2.3                   | <b>100.6 <math>\pm</math> 12.8</b> | 8.1 $\pm$ 1.3                   | <b>0.6 <math>\pm</math> 0.0</b> | 157.9 $\pm$ 18.8                  | <b>55.8</b> |
| FDR          | 40.2 $\pm$ 2.9                   | 256.3 $\pm$ 12.1                 | 806.2 $\pm$ 13.0                   | 1.6 $\pm$ 0.0                   | 0.7 $\pm$ 0.0                   | 161.4 $\pm$ 14.6                  | 211.1       |
| ER-ACE       | 55.2 $\pm$ 14.2                  | 350.5 $\pm$ 21.1                 | 115.2 $\pm$ 10.5                   | 1.5 $\pm$ 0.0                   | 1.1 $\pm$ 0.0                   | 289.8 $\pm$ 77.0                  | 135.5       |
| LODE         | 30.6 $\pm$ 1.9                   | 233.4 $\pm$ 13.3                 | 776.7 $\pm$ 17.2                   | <b>1.4 <math>\pm</math> 0.0</b> | 0.6 $\pm$ 0.0                   | 180.7 $\pm$ 17.2                  | 203.9       |
| STAR         | 66.7 $\pm$ 17.0                  | 627.9 $\pm$ 49.9                 | 872.2 $\pm$ 25.0                   | 1.9 $\pm$ 0.0                   | 0.9 $\pm$ 0.1                   | 158.5 $\pm$ 16.2                  | 288.0       |
| COPE         | 48.2 $\pm$ 9.3                   | 1604.8 $\pm$ 410.3               | 958.4 $\pm$ 100.7                  | 4.6 $\pm$ 0.3                   | 7.2 $\pm$ 0.2                   | 222.2 $\pm$ 50.2                  | 474.2       |
| EFC          | 39.9 $\pm$ 4.5                   | 301.5 $\pm$ 25.5                 | 195.0 $\pm$ 21.5                   | 2.5 $\pm$ 0.3                   | 0.8 $\pm$ 0.0                   | 205.8 $\pm$ 27.4                  | 124.2       |
| SARL         | 55.8 $\pm$ 15.7                  | 367.9 $\pm$ 23.2                 | 829.7 $\pm$ 6.4                    | 1.7 $\pm$ 0.0                   | 0.8 $\pm$ 0.1                   | 166.8 $\pm$ 18.7                  | 237.1       |
| ★-CL-VREX    | 77.5 $\pm$ 14.8                  | 588.0 $\pm$ 78.3                 | 875.3 $\pm$ 12.5                   | 1.8 $\pm$ 0.0                   | 5.0 $\pm$ 0.2                   | 180.6 $\pm$ 15.7                  | 288.0       |
| ★-CL-Fishr   | 58.3 $\pm$ 1.8                   | 423.9 $\pm$ 8.5                  | 974.0 $\pm$ 18.0                   | 2.2 $\pm$ 0.0                   | 5.0 $\pm$ 0.1                   | 157.1 $\pm$ 19.2                  | 270.1       |
| ★-CL-CORAL   | 48.0 $\pm$ 2.3                   | 484.3 $\pm$ 95.3                 | 794.2 $\pm$ 11.3                   | 1.9 $\pm$ 0.3                   | 5.4 $\pm$ 0.4                   | 318.2 $\pm$ 106.9                 | 275.3       |
| ★-CL-MMD     | 48.1 $\pm$ 2.9                   | 598.3 $\pm$ 9.0                  | 875.7 $\pm$ 5.1                    | 2.0 $\pm$ 0.3                   | 4.7 $\pm$ 0.0                   | 196.5 $\pm$ 14.3                  | 287.6       |
| ★-CL-ANDMask | 55.3 $\pm$ 4.9                   | 294.7 $\pm$ 10.3                 | 887.3 $\pm$ 9.5                    | 2.2 $\pm$ 0.1                   | 4.9 $\pm$ 0.2                   | 219.8 $\pm$ 52.2                  | 244.0       |

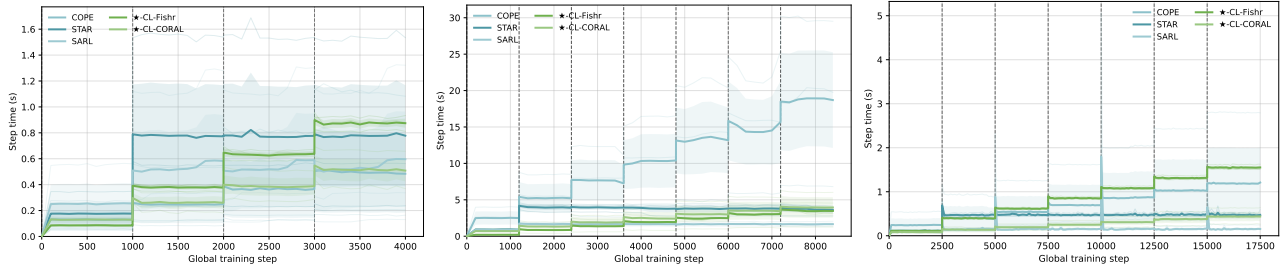


Figure 5. **Average per-step runtimes over time.** We plot the average per-step runtime for methods COPE, STAR, SARL, ★-CL-Fishr and ★-CL-CORAL across datasets RotatedMNIST (left), CIFAR10C (middle), and TinyImageNetC (right).

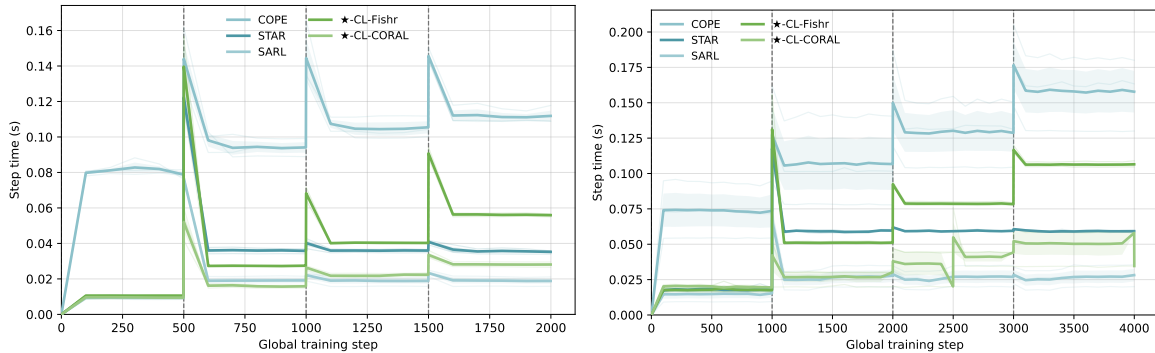


Figure 6. **Average per-step runtimes over time.** We plot the average per-step runtime for methods COPE, STAR, SARL, ★-CL-Fishr and ★-CL-CORAL across datasets RotatedMNIST (left), CIFAR10C (middle), and TinyImageNetC (right).

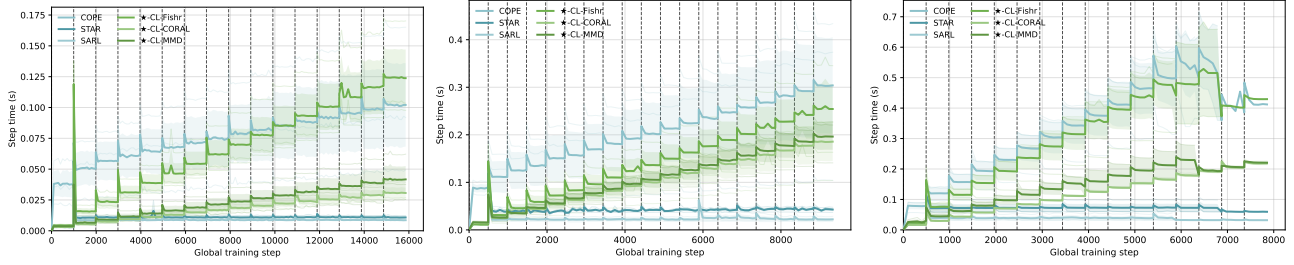


Figure 7. **Average per-step runtimes over time (extended datasets).** We plot the average per-step runtime for methods COPE, STAR, SARL, ★-CL-Fishr and ★-CL-CORAL across extended datasets (15 source domains) RotatedMNISTExtended (left), WM811KExtended (middle), and Camelyon17Extended (right).

### G.6. Ablation of the invariance alignment

We here present the results for two experiments: (1) We disable the invariance alignment entirely, by setting  $\beta$  to 0<sup>4</sup>. (2) We dynamically recompute the anchor statistics  $\Phi_{s'}$  for previous domains at the end of the current domain. The results for the ablations can be found in Tables 10 and 11. We find that *anchoring to the original (unaltered) representations generally improves the performance*.

Table 10. Results for disabling the sequential invariance alignment. Shown: mean  $\pm$  standard error target domain accuracy across three independent runs.

| Ablation       | Method     | RotatedMNIST   | Coverttype     | Avg  |
|----------------|------------|----------------|----------------|------|
| W/o alignment  | CL-VREX    | 62.4 $\pm$ 1.3 | 37.1 $\pm$ 0.4 | 49.7 |
|                | CL-Fishr   | 74.7 $\pm$ 4.5 | 37.5 $\pm$ 0.8 | 56.1 |
|                | CL-CORAL   | 69.5 $\pm$ 1.3 | 37.5 $\pm$ 1.6 | 53.5 |
|                | CL-MMD     | 66.5 $\pm$ 0.5 | 37.6 $\pm$ 1.0 | 52.0 |
|                | CL-ANDMask | 70.8 $\pm$ 4.1 | 34.2 $\pm$ 1.0 | 52.5 |
| With alignment | CL-VREX    | 70.2 $\pm$ 5.8 | 40.8 $\pm$ 1.5 | 55.5 |
|                | CL-Fishr   | 68.6 $\pm$ 0.4 | 27.0 $\pm$ 7.7 | 47.8 |
|                | CL-CORAL   | 72.8 $\pm$ 2.6 | 45.2 $\pm$ 3.9 | 59.0 |
|                | CL-MMD     | 70.7 $\pm$ 0.5 | 37.6 $\pm$ 0.9 | 54.2 |
|                | CL-ANDMask | 71.4 $\pm$ 2.6 | 43.7 $\pm$ 2.9 | 57.6 |

<sup>4</sup>Note that ablating the effect of the invariance penalty itself (i.e., Equation (6)) would simultaneously deactivate the alignment loss, as there would not be any statistics to align. Thus, this would fall back to normal fine-tuning with a replay buffer. The results for this “non-ablation” can be found as the ER-ACE method, which essentially uses replay with sequential fine-tuning, in Table 2.

Table 11. **Ablating the effect of dynamically recomputing  $\Phi_{s'}$ .** We recompute  $\Phi_{s'}$  for previous domains at the end of the current domain, using the data stored in the replay buffer. For comparison, we give the results for fixing the anchor representations (“Static  $\Phi_{s'}$ ”). We report the mean accuracy (macro F1 for WM811K)  $\pm$  standard error *target domain performance* across three independent runs.

|                     | Method     | RotatedMNIST   | CIFAR10C       | WM811K         | Coverttype     | Camelyon17     | Avg  |
|---------------------|------------|----------------|----------------|----------------|----------------|----------------|------|
| Static $\Phi_{s'}$  | ★-CL-VREX  | 70.2 $\pm$ 5.8 | 67.2 $\pm$ 1.4 | 84.1 $\pm$ 0.3 | 40.8 $\pm$ 1.5 | 91.5 $\pm$ 0.6 | 70.8 |
|                     | ★-CL-Fishr | 68.6 $\pm$ 0.4 | 64.2 $\pm$ 1.2 | 83.6 $\pm$ 0.5 | 27.0 $\pm$ 7.7 | 89.7 $\pm$ 2.0 | 66.6 |
|                     | ★-CL-CORAL | 72.8 $\pm$ 2.6 | 68.5 $\pm$ 1.7 | 84.8 $\pm$ 0.6 | 45.2 $\pm$ 3.9 | 91.7 $\pm$ 0.4 | 72.6 |
|                     | ★-CL-MMD   | 70.7 $\pm$ 0.5 | 69.0 $\pm$ 0.5 | 85.5 $\pm$ 0.4 | 37.6 $\pm$ 0.9 | 90.1 $\pm$ 0.5 | 70.6 |
| Dynamic $\Phi_{s'}$ | ★-CL-VREX  | 71.8 $\pm$ 6.0 | 65.6 $\pm$ 1.3 | 74.0 $\pm$ 1.7 | 36.5 $\pm$ 0.6 | 90.7 $\pm$ 0.3 | 67.7 |
|                     | ★-CL-Fishr | 67.9 $\pm$ 1.1 | 22.9 $\pm$ 4.0 | 76.3 $\pm$ 0.7 | 29.9 $\pm$ 6.0 | 89.9 $\pm$ 1.5 | 57.4 |
|                     | ★-CL-CORAL | 74.7 $\pm$ 1.0 | 66.3 $\pm$ 1.3 | 73.6 $\pm$ 1.5 | 33.9 $\pm$ 0.6 | 88.8 $\pm$ 0.0 | 67.5 |
|                     | ★-CL-MMD   | 68.3 $\pm$ 0.4 | 66.9 $\pm$ 0.5 | 71.6 $\pm$ 1.9 | 38.6 $\pm$ 1.3 | 89.4 $\pm$ 1.0 | 67.0 |

### G.7. Robustness check: reduced buffer sizes

We here present the full results for our robustness experiments that reduce the buffer capacity to 50 % (in Table 12) and 25 % (in Table 13).

Table 12. Results for a 50 % reduced buffer size. Shown: mean  $\pm$  standard error target domain accuracy across three independent runs. Marked: **best**, second, third.

| Method       | Accuracy ( $\uparrow$ )          |                                  |                                  |             | Rank ( $\downarrow$ ) |            |            |
|--------------|----------------------------------|----------------------------------|----------------------------------|-------------|-----------------------|------------|------------|
|              | RotatedMNIST                     | Coverttype                       | CIFAR10C                         | Avg         | Arith. mean           | Geom. mean | Median     |
| FDR          | 57.8 $\pm$ 0.6                   | <u>65.8 <math>\pm</math> 0.1</u> | 37.7 $\pm$ 0.9                   | 53.8        | <u>4.0</u>            | <u>3.6</u> | <u>4.0</u> |
| ER-ACE       | 56.1 $\pm$ 2.0                   | 64.4 $\pm$ 0.8                   | <u>38.1 <math>\pm</math> 0.6</u> | 52.8        | 5.7                   | 5.3        | 7.0        |
| STAR         | 55.6 $\pm$ 2.5                   | <u>65.6 <math>\pm</math> 0.9</u> | <b>39.0 <math>\pm</math> 1.1</b> | 53.4        | <u>4.0</u>            | <u>2.9</u> | <u>3.0</u> |
| ★-CL-VREX    | <b>68.1 <math>\pm</math> 1.0</b> | <b>66.2 <math>\pm</math> 1.0</b> | 38.4 $\pm$ 1.3                   | <b>57.5</b> | <b>1.3</b>            | <b>1.3</b> | <b>1.0</b> |
| ★-CL-Fishr   | <u>67.5 <math>\pm</math> 0.7</u> | 64.4 $\pm$ 0.8                   | 29.2 $\pm$ 7.6                   | 53.7        | 5.7                   | 5.2        | 6.0        |
| ★-CL-CORAL   | 66.0 $\pm$ 3.5                   | 65.3 $\pm$ 1.2                   | 37.3 $\pm$ 0.7                   | <u>56.2</u> | <u>4.7</u>            | 4.6        | <u>4.0</u> |
| ★-CL-MMD     | 64.9 $\pm$ 7.6                   | 65.3 $\pm$ 0.5                   | 37.5 $\pm$ 0.7                   | <u>55.9</u> | 5.0                   | 5.0        | 5.0        |
| ★-CL-ANDMask | <u>67.5 <math>\pm</math> 1.2</u> | 62.0 $\pm$ 0.6                   | 36.0 $\pm$ 1.3                   | 55.2        | 5.7                   | 4.8        | 7.0        |

Table 13. Results for a 75 % reduced buffer size. Shown: mean $\pm$ standard error target domain accuracy across three independent runs. Marked: **best**, second, third.

| Method       | RotatedMNIST                     | Accuracy ( $\uparrow$ )          |                                  |             | Avg        | Rank ( $\downarrow$ ) |            |            |
|--------------|----------------------------------|----------------------------------|----------------------------------|-------------|------------|-----------------------|------------|------------|
|              |                                  | Coverttype                       | CIFAR10C                         |             |            | Arith. mean           | Geom. mean | Median     |
| FDR          | 62.5 $\pm$ 2.7                   | 64.5 $\pm$ 0.6                   | 37.3 $\pm$ 0.5                   | <u>54.8</u> | 5.3        | 5.3                   | 5.0        | 5.0        |
| ER-ACE       | 54.0 $\pm$ 2.1                   | 63.7 $\pm$ 0.5                   | <u>37.6 <math>\pm</math> 0.4</u> | 51.8        | 5.3        | 5.0                   | 6.0        | 6.0        |
| STAR         | 52.9 $\pm$ 2.6                   | <u>66.6 <math>\pm</math> 0.4</u> | <u>37.7 <math>\pm</math> 0.7</u> | 52.4        | <u>4.0</u> | <u>3.2</u>            | <u>2.0</u> | <u>2.0</u> |
| ★-CL-VREX    | <b>67.1 <math>\pm</math> 8.1</b> | <b>67.0 <math>\pm</math> 0.6</b> | <b>41.1 <math>\pm</math> 2.8</b> | <b>58.4</b> | <b>1.0</b> | <b>1.0</b>            | <b>1.0</b> | <b>1.0</b> |
| ★-CL-Fishr   | 63.9 $\pm$ 1.9                   | 63.3 $\pm$ 1.1                   | 28.7 $\pm$ 6.9                   | 52.0        | 6.3        | 6.1                   | 7.0        | 7.0        |
| ★-CL-CORAL   | <u>65.5 <math>\pm</math> 2.7</u> | 65.0 $\pm$ 1.6                   | 37.5 $\pm$ 2.1                   | <u>56.0</u> | <u>3.3</u> | <u>3.2</u>            | <u>4.0</u> | <u>4.0</u> |
| ★-CL-MMD     | 55.5 $\pm$ 3.0                   | <u>65.1 <math>\pm</math> 1.0</u> | 37.4 $\pm$ 0.9                   | 52.7        | 4.7        | <u>4.5</u>            | 5.0        | 5.0        |
| ★-CL-ANDMask | <u>64.5 <math>\pm</math> 1.4</u> | 60.6 $\pm$ 1.0                   | 35.8 $\pm$ 1.3                   | 53.7        | 6.0        | 5.5                   | 7.0        | 7.0        |

### G.8. Source-domain performance

In this section, we discuss how we report the performance on the source domain sequence. Recall that, in continual learning, a model is sequentially trained on a sequence of source (training) domains. Commonly, the models are evaluated w.r.t. their ability to maintain or even retrospectively improve the performance across these training domains. For this evaluation, the CL literature utilizes a hold-out subset of the training distribution. This is related to *in-domain generalization*, which evaluates the performance of a model on an IID set of the training distribution (c.f. [Hardt et al., 2016](#)).

In contrast, in our paper we are interested in the post-training performance, that is, the ability of continually trained models to generalize to an unseen target domain. This is relevant in field such as manufacturing or medicine, where storage or privacy concerns disallow retaining or sharing the training data, yet models need to generalize to new products (in manufacturing) or hospitals (in medicine) after deployment. As such, after continual training on the source-domain sequence, we primarily evaluate the performance on an unseen target domain. Due to this setting, our reporting of the source performance is thus slightly different to the CL literature. In the CL literature, the best runs are selected based on validation splits of the training distributions, and metrics are reported for the test splits of the selected runs (ie, in-domain generalization). In our setting, we equally select the best runs based on the validation splits (see Section F), but differently report metrics for a novel distribution (out-of-domain generalization). As such, no test split of the training distributions is required for our setting, and hence not available

To report the performance on the source domains, we thus **have three options**: **Option 1**: select the best runs as used throughout the paper (as described in Section F) and report the validation-split source performance. This directly gives the metrics used to select the best runs w.r.t. target domain performance, i.e., the runs reported in the tables throughout this paper. However, this is basically optimization on the test set (cf. [Dwork et al., 2015](#); [Shankar et al., 2021](#)). **Option 2**: select the best runs as in Option 1, but report the training-split source performance. **Option 3**: select the best runs based on the training-splits of the source domains, and report the validation-split performance. However, the selected runs are then not related to the runs reported in this paper

Given these three options, we can directly discard the last one as it essentially studies a distinct setting. From the remaining options, Option 2 is the most sensible one, as it avoids reporting results that are optimized on the test set. Additionally, Option 2 reports the performance on the data that was actually used during continually training on the source domains. The results are in Table 14.

Table 14. Source domain performance; computed as described in Section G.8. Shown: mean $\pm$ standard error source domain accuracy (macro F1 for WM811K) across three independent runs.

| Method       | RotatedMNIST   | CIFAR10C       | TinyImageNetC   | WM811K         | Coverttype     | Camelyon17     | Avg  |
|--------------|----------------|----------------|-----------------|----------------|----------------|----------------|------|
| Finetune     | 96.5 $\pm$ 0.2 | 84.5 $\pm$ 1.0 | 67.2 $\pm$ 0.9  | 91.6 $\pm$ 0.3 | 64.3 $\pm$ 0.5 | 92.4 $\pm$ 0.7 | 82.7 |
| AGEM         | 97.7 $\pm$ 0.3 | 90.1 $\pm$ 1.2 | 72.3 $\pm$ 0.9  | 91.6 $\pm$ 0.7 | 67.3 $\pm$ 0.3 | 94.7 $\pm$ 0.2 | 85.6 |
| UPGD         | 85.0 $\pm$ 2.3 | 63.7 $\pm$ 6.4 | 46.8 $\pm$ 8.0  | 75.0 $\pm$ 1.6 | 54.1 $\pm$ 3.5 | 91.4 $\pm$ 0.6 | 69.3 |
| EWC          | 96.7 $\pm$ 0.3 | 86.2 $\pm$ 0.7 | 67.5 $\pm$ 1.5  | 92.2 $\pm$ 0.2 | 64.6 $\pm$ 1.4 | 92.6 $\pm$ 0.2 | 83.3 |
| SI           | 96.1 $\pm$ 0.4 | 82.4 $\pm$ 0.9 | 65.9 $\pm$ 1.2  | 91.4 $\pm$ 1.7 | 64.5 $\pm$ 0.7 | 92.8 $\pm$ 0.6 | 82.2 |
| SNR          | 96.5 $\pm$ 0.4 | 82.0 $\pm$ 0.3 | 66.1 $\pm$ 0.6  | 77.7 $\pm$ 1.0 | 62.4 $\pm$ 1.6 | 91.7 $\pm$ 0.3 | 79.4 |
| FDR          | 98.9 $\pm$ 0.1 | 88.9 $\pm$ 0.5 | 83.2 $\pm$ 0.3  | 92.2 $\pm$ 0.4 | 80.0 $\pm$ 0.6 | 94.8 $\pm$ 0.0 | 89.7 |
| ER-ACE       | 99.1 $\pm$ 0.0 | 91.1 $\pm$ 1.1 | 87.3 $\pm$ 1.7  | 93.8 $\pm$ 0.4 | 81.5 $\pm$ 0.8 | 95.4 $\pm$ 0.2 | 91.4 |
| LODE         | 96.8 $\pm$ 0.2 | 88.0 $\pm$ 0.5 | 64.1 $\pm$ 3.1  | 86.3 $\pm$ 1.3 | 67.9 $\pm$ 1.9 | 70.1 $\pm$ 7.7 | 78.9 |
| STAR         | 98.8 $\pm$ 0.0 | 93.1 $\pm$ 0.2 | 83.5 $\pm$ 0.1  | 94.4 $\pm$ 0.5 | 80.2 $\pm$ 0.4 | 95.4 $\pm$ 0.0 | 90.9 |
| COPE         | 97.7 $\pm$ 0.1 | 88.6 $\pm$ 1.4 | 45.8 $\pm$ 10.4 | 91.0 $\pm$ 1.1 | 2.0 $\pm$ 1.0  | 94.6 $\pm$ 0.2 | 69.9 |
| EFC          | 98.5 $\pm$ 0.2 | 89.8 $\pm$ 1.2 | 73.9 $\pm$ 4.2  | 93.6 $\pm$ 1.3 | 66.3 $\pm$ 2.3 | 94.7 $\pm$ 0.2 | 86.1 |
| SARL         | 98.8 $\pm$ 0.1 | 84.5 $\pm$ 3.1 | 0.7 $\pm$ 0.1   | 82.1 $\pm$ 1.6 | 75.4 $\pm$ 0.5 | 84.7 $\pm$ 0.6 | 71.0 |
| ★-CL-VREX    | 98.6 $\pm$ 0.1 | 89.7 $\pm$ 0.5 | 83.5 $\pm$ 0.8  | 94.7 $\pm$ 0.5 | 72.9 $\pm$ 1.1 | 95.3 $\pm$ 0.2 | 89.1 |
| ★-CL-Fishr   | 99.3 $\pm$ 0.0 | 81.3 $\pm$ 1.3 | 85.9 $\pm$ 2.6  | 96.4 $\pm$ 0.1 | 63.1 $\pm$ 1.3 | 94.9 $\pm$ 0.1 | 86.8 |
| ★-CL-CORAL   | 99.1 $\pm$ 0.0 | 91.2 $\pm$ 1.0 | 85.7 $\pm$ 1.9  | 93.6 $\pm$ 1.0 | 63.2 $\pm$ 0.7 | 95.4 $\pm$ 0.1 | 88.0 |
| ★-CL-MMD     | 99.0 $\pm$ 0.1 | 91.1 $\pm$ 0.5 | 83.7 $\pm$ 0.5  | 93.8 $\pm$ 1.0 | 73.5 $\pm$ 0.7 | 95.6 $\pm$ 0.0 | 89.5 |
| ★-CL-ANDMask | 99.0 $\pm$ 0.1 | 88.9 $\pm$ 0.4 | 36.3 $\pm$ 4.1  | 93.8 $\pm$ 0.2 | 70.0 $\pm$ 1.7 | 94.4 $\pm$ 0.2 | 80.4 |

### G.9. Robustness check: different target domains

For our main experiments, we fix the target domain per dataset as indicated in Section E. For this robustness experiment, we change the target domain. For RotatedMNIST, we select  $D^0$ ; for CIFAR10C, we select  $D^{\text{brightness+contrast}}$ , and for Coverttype we select  $D^{\text{Rawah}}$ . The new target domains are thus all from the former set of source domains; hence, a new hyperparameter study is required. The results are given in Table 15.

Table 15. Results for different target domains. Shown: mean $\pm$ standard error target domain accuracy across three independent runs. Marked: **best**, second, third.

| Method       | RotatedMNIST                     | CIFAR10C                         | Coverttype                       | Avg         |
|--------------|----------------------------------|----------------------------------|----------------------------------|-------------|
| Finetune     | 54.8 $\pm$ 1.1                   | 49.7 $\pm$ 1.6                   | <b>49.2 <math>\pm</math> 2.8</b> | 51.2        |
| AGEM         | 65.5 $\pm$ 2.3                   | 49.8 $\pm$ 0.3                   | 35.5 $\pm$ 1.4                   | 50.3        |
| UPGD         | 34.2 $\pm$ 1.9                   | 39.4 $\pm$ 0.5                   | <u>46.6 <math>\pm</math> 4.1</u> | 40.1        |
| EWC          | 60.4 $\pm$ 2.4                   | 49.6 $\pm$ 1.5                   | 40.4 $\pm$ 1.9                   | 50.1        |
| SI           | 53.4 $\pm$ 1.9                   | 50.0 $\pm$ 1.0                   | 35.4 $\pm$ 1.6                   | 46.3        |
| FDR          | 86.0 $\pm$ 0.5                   | <b>52.5 <math>\pm</math> 1.2</b> | 39.3 $\pm$ 1.7                   | <u>59.3</u> |
| ER-ACE       | <u>88.4 <math>\pm</math> 3.3</u> | 45.7 $\pm$ 1.5                   | 40.2 $\pm$ 3.3                   | 58.1        |
| STAR         | 84.6 $\pm$ 0.3                   | <u>51.7 <math>\pm</math> 0.4</u> | 39.4 $\pm$ 1.1                   | 58.6        |
| ★-CL-VREX    | 85.9 $\pm$ 1.6                   | <u>50.4 <math>\pm</math> 1.2</u> | 38.3 $\pm$ 1.9                   | 58.2        |
| ★-CL-Fishr   | <b>91.8 <math>\pm</math> 0.4</b> | 49.6 $\pm$ 1.8                   | 35.7 $\pm$ 3.0                   | <u>59.1</u> |
| ★-CL-CORAL   | 87.1 $\pm$ 0.8                   | 48.2 $\pm$ 1.0                   | 38.5 $\pm$ 1.4                   | 57.9        |
| ★-CL-MMD     | 85.7 $\pm$ 1.2                   | 49.0 $\pm$ 0.9                   | <u>42.4 <math>\pm</math> 1.0</u> | 59.0        |
| ★-CL-ANDMask | <u>89.6 <math>\pm</math> 0.6</u> | 49.6 $\pm$ 0.7                   | 40.4 $\pm$ 1.1                   | <b>59.8</b> |

**G.10. Robustness: different compute limit**

For our main experiments, we fix the compute budget, which is the number of training steps per domain. We here set the compute budget to 50 % of the original value. Result are in Table 16.

Table 16. Results for a reduced compute budget. Shown: mean $\pm$ standard error target domain accuracy across three independent runs. Marked: **best**, second, third.

| Method       | RotatedMNIST                     | CIFAR10C                         | Coverttype                       | Avg         |
|--------------|----------------------------------|----------------------------------|----------------------------------|-------------|
| Finetune     | 39.4 $\pm$ 1.7                   | 58.4 $\pm$ 1.5                   | 27.9 $\pm$ 5.8                   | 41.9        |
| AGEM         | 48.7 $\pm$ 4.7                   | 64.1 $\pm$ 0.8                   | 36.9 $\pm$ 0.5                   | 49.9        |
| UPGD         | 26.6 $\pm$ 6.3                   | 55.0 $\pm$ 5.6                   | 18.6 $\pm$ 8.0                   | 33.4        |
| EWC          | 33.8 $\pm$ 0.4                   | 61.3 $\pm$ 0.8                   | 29.8 $\pm$ 5.9                   | 41.6        |
| SI           | 36.7 $\pm$ 0.8                   | 60.4 $\pm$ 0.5                   | <b>41.8 <math>\pm</math> 3.2</b> | 46.3        |
| FDR          | 68.4 $\pm$ 0.9                   | 64.2 $\pm$ 0.8                   | 38.9 $\pm$ 0.1                   | 57.2        |
| ER-ACE       | 63.8 $\pm$ 3.1                   | 63.8 $\pm$ 0.9                   | 37.3 $\pm$ 0.2                   | 55.0        |
| STAR         | 67.2 $\pm$ 0.6                   | 65.0 $\pm$ 0.7                   | 37.8 $\pm$ 0.6                   | 56.7        |
| ★-CL-VREX    | 69.4 $\pm$ 5.2                   | 64.5 $\pm$ 0.9                   | 37.9 $\pm$ 0.7                   | 57.2        |
| ★-CL-Fishr   | 64.0 $\pm$ 2.0                   | 60.1 $\pm$ 2.9                   | 34.8 $\pm$ 1.4                   | 53.0        |
| ★-CL-CORAL   | <b>73.1 <math>\pm</math> 3.3</b> | <b>66.0 <math>\pm</math> 1.2</b> | 36.4 $\pm$ 1.1                   | <b>58.5</b> |
| ★-CL-MMD     | 67.4 $\pm$ 2.3                   | 64.2 $\pm$ 0.7                   | 36.7 $\pm$ 0.4                   | 56.1        |
| ★-CL-ANDMask | 66.4 $\pm$ 0.2                   | 63.9 $\pm$ 0.5                   | 33.9 $\pm$ 2.9                   | 54.7        |

### G.11. Visualizations: Hyperparameter sensitivity

We here analyze the sensitivity of our proposed methods to changes in the  $\lambda$  and  $\beta$  parameters. For this, we select the best run of the three runs, and vary  $\lambda, \beta$  across 10 different values picked from their value range in Table 4. We perform this analysis on three different datasets, namely, RotatedMNIST, WM811K, Covertypes. The sensitivity plots are in Figures 8 and 9. We find that our proposed  $\star$ -CL methods are relatively robust, with changes often being less than 2pp.

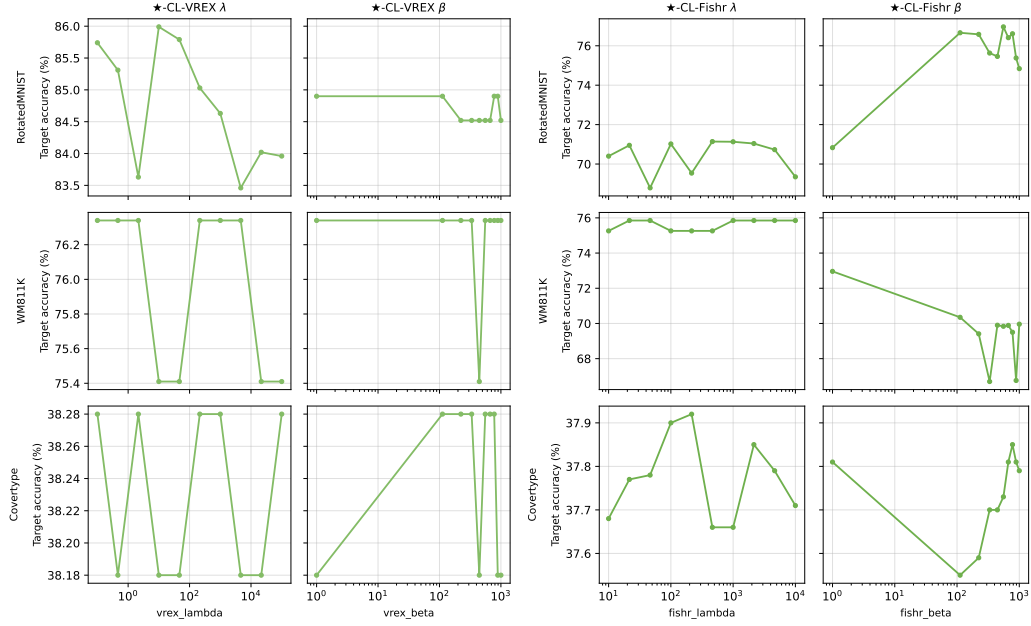


Figure 8. **Hyperparameter sensitivity plots (I).** We ablate the effect of varying  $\lambda$  and  $\beta$  parameters for  $\star$ -CL-VREX (left) and  $\star$ -CL-Fishr (right).

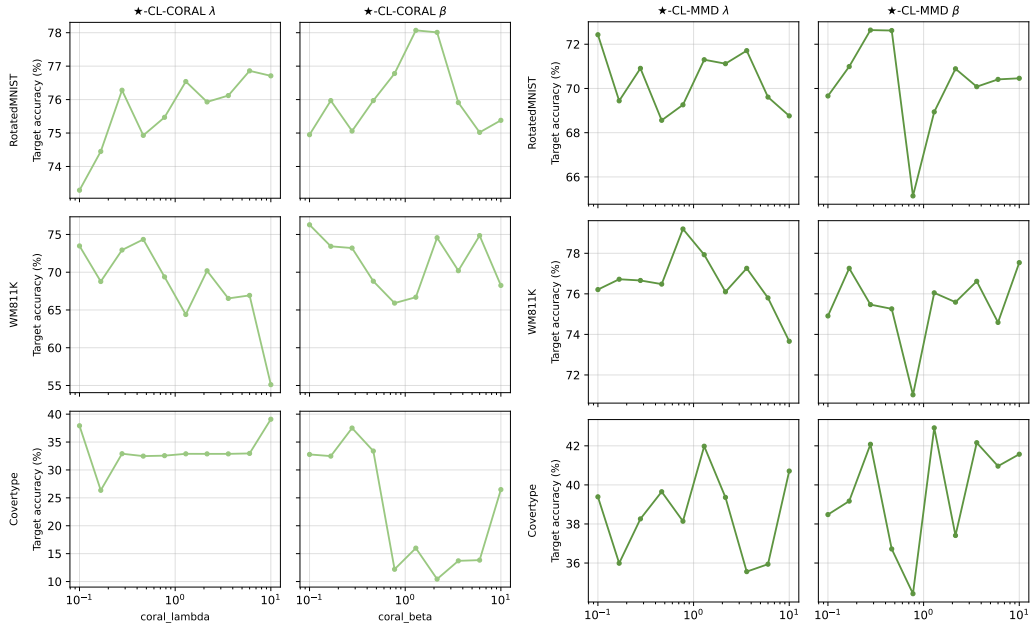


Figure 9. **Hyperparameter sensitivity plots (II).** We ablate the effect of varying  $\lambda$  and  $\beta$  parameters for  $\star$ -CL-CORAL (left) and  $\star$ -CL-MMD (right).

## G.12. Visualizations: feature spaces

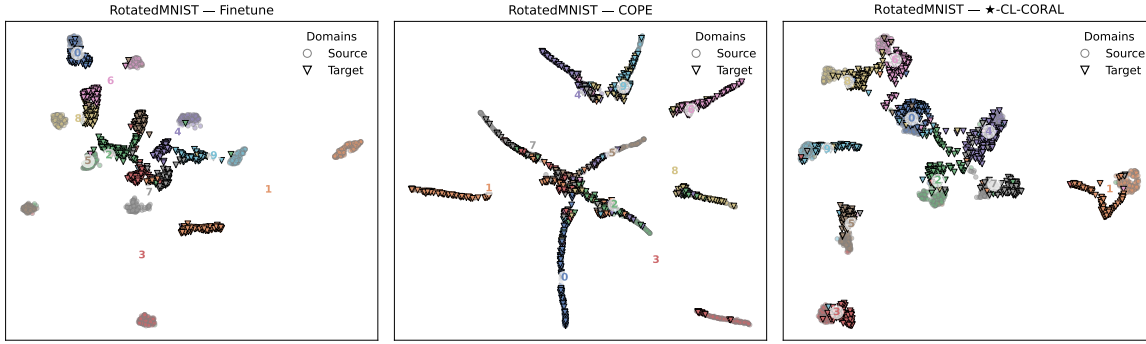


Figure 10. **Feature space visualizations (RotatedMNIST).** We visualize the feature space of Finetune (left), COPE (middle), and ★-CL-CORAL (right) using UMAP (McInnes et al., 2018). Colored numbers denote cluster centroids for the respective class.

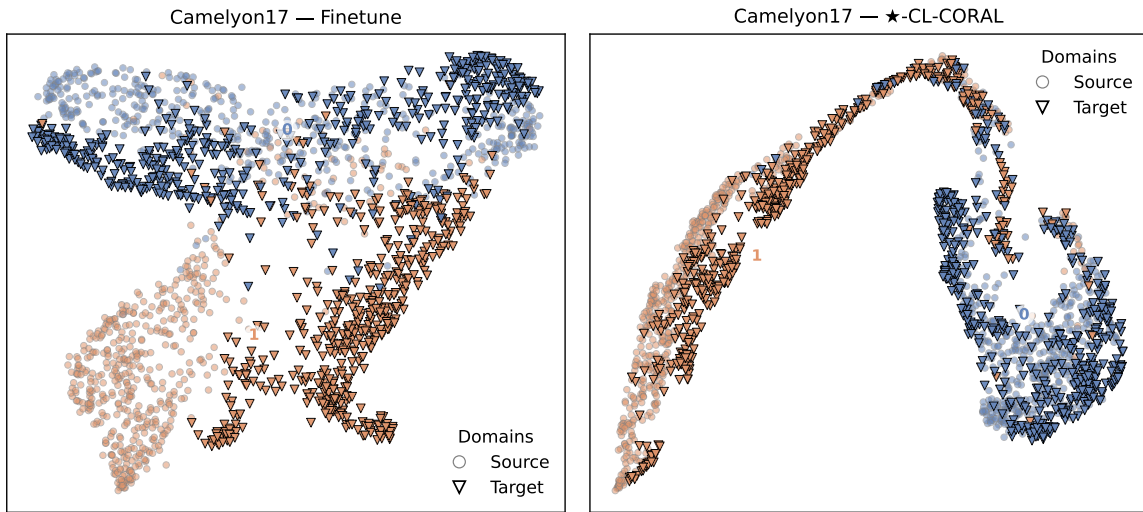


Figure 11. **Feature space visualizations (Camelyon17).** We visualize the feature space of Finetune (left) and ★-CL-CORAL (right) using UMAP (McInnes et al., 2018). Colored numbers denote cluster centroids for the respective class.